

Beyond Test-Time Memory: State-Space Optimal Control for LLM Reasoning

Peihao Wang¹ Shan Yang² Xijun Wang² Tesi Xiao² Xin Liu² Changlong Yu² Yu Lou²
Pan Li³ Zhangyang Wang¹ Ming Lin⁴ René Vidal⁵

 vita-group.github.io/TTC-Net

Abstract

Associative memory has long underpinned the design of sequential models. Beyond recall, humans reason by *projecting future states and selecting goal-directed actions*, a capability that modern language models increasingly require but do not natively encode. While prior work uses reinforcement learning or test-time training, planning remains external to the model architecture. We formulate reasoning as *optimal control* and introduce the *Test-Time Control (TTC)* layer, which performs finite-horizon *LQR* planning over latent states at inference time, represents a *value function* within neural architectures, and leverages it as the nested objective to enable *planning before prediction*. To ensure scalability, we derive a hardware-efficient LQR solver based on a symplectic formulation and implement it as a fused CUDA kernel, enabling parallel execution with minimal overhead. Integrated as an adapter into pretrained LLMs, TTC layers improve mathematical reasoning performance by up to **+27.8%** on MATH-500 and **2-3 $\times$** Pass@8 improvements on AMC and AIME, demonstrating that embedding optimal control as an architectural component provides an effective and scalable mechanism for reasoning beyond test-time training.

1. Introduction

Sequence-processing architectures have evolved from recurrent neural networks (RNNs) (Hochreiter & Schmidhuber,

¹University of Texas at Austin ²Amazon ³Georgia Institute of Technology ⁴University of Maryland at College Park ⁵University of Pennsylvania. Correspondence to: Peihao Wang <peihao.wang@utexas.edu>.

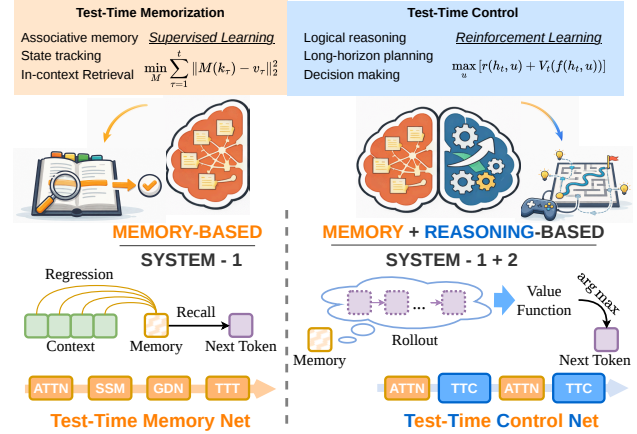


Figure 1. Memory-only prediction vs. unified memory-control planning. Memory-based models resemble human System 1 processing, relying on associative retrieval and producing fixed responses. Inspired by System 2 cognition, we model reasoning as an explicit optimal control problem and propose **TTC-Net**, a unified architecture that integrates test-time control (TTC) layers to encode value functions during sequential modeling, enabling planning before prediction.

1997; Sutskever et al., 2014; Cho et al., 2014) to transformers (Vaswani et al., 2017) and, more recently, to State-Space Models (SSMs) and linear RNNs (Gu & Dao, 2023; Yang et al., 2023b; 2024a; Beck et al., 2024). Despite substantial architectural differences, these models share a common design principle: prediction through *associative memory*. Past context is encoded as memory states, and the next token is generated by retrieving or decoding information from this stored representation (Hopfield, 1982; Hopfield & Tank, 1985; Widrich et al., 2020; Wang et al., 2025b). Attention mechanisms explicitly retain all past key-value pairs and match queries against them (Sun et al., 2024), while linear RNNs and SSMs progressively compress historical context into a fixed-size latent state via online regression and decode this state to produce future predictions (Schlag et al., 2021; Yang et al., 2024b; Behrouz et al., 2024; 2025a;c).

This memory-centric paradigm has proven highly effective for language modeling, yet increasingly reveals limitations when models are required to *reason*, *discover*, or *solve prob-*

lems. These tasks demand mechanisms beyond memorization and retrieval. From a cognitive perspective, human intelligence operates through an interplay between System 1 and System 2 thinking (Kahneman, 2011). System 1 relies on fast, automatic pattern matching over memory, while System 2 engages in deliberate, multi-step planning and long-horizon reasoning. Current LLM architectures largely instantiate System 1 behavior: they predict the next token by extrapolating from past context, but lack a dedicated architectural mechanism for System 2-style planning.

Recent advances in reinforcement learning (RL) have partially addressed this gap by making language models more goal-directed and planning-oriented, particularly in mathematical reasoning and coding (Shao et al., 2024; Guo et al., 2025). However, RL is typically applied as an external training or post-training procedure. The reward-driven optimization is absent during early pretraining and remains decoupled from the model’s core inference mechanism (Hatamizadeh et al., 2025; Kobayashi et al., 2025). As a result, RL alone cannot fully overcome the reasoning ceilings imposed by memory-based architectures (Yue et al., 2025). The model learns *what* to optimize, but not *how* to reason through planning as part of its forward computation.

1.1. Main Contributions

Motivated by this gap in reasoning, we propose a different architectural perspective: ***viewing reasoning and planning as an optimal control problem over internal representations***. Rather than treating planning as a training-time or test-time optimization procedure, we internalize it directly into the model architecture. We operationalize this perspective architecturally by introducing a *Test-Time Control (TTC)* layer, which maps memory representations to optimal actions of a tractable control problem. Given a context-encoded latent state, a TTC layer performs test-time planning by solving a Markov Decision Process (MDP) with linear state transitions and quadratic cost functions, corresponding to a *Linear-Quadratic Regulator (LQR)*. The first-step optimal control action is then decoded as the next-token representation, enabling the model to *plan before prediction*.

Crucially, the TTC layer performs *online world modeling* (Ha & Schmidhuber, 2018) and endows each language modeling block with a latent *value function*. Environment dynamics and cost functions are synthesized from context, allowing planning to adapt to the current reasoning state. To capture temporal structure and long-horizon objectives, the TTC layer supports time-heterogeneous parameterizations of both dynamics and costs. Existing fast-weight test-time memorization methods (Yang et al., 2024b; Sun et al., 2024; Behrouz et al., 2025b) interpret inference as test-time estimation, solving self-supervised regression problems to better encode past context. In contrast, TTC interprets in-

ference as test-time decision making, solving a structured optimal control problem to select actions that optimize future outcomes.

To enable end-to-end learning, we derive a differentiable formulation of the TTC layer inspired by differentiable optimization (Amos & Kolter, 2017). By casting the LQR problem into a KKT system, gradients can be propagated through the optimal solution by solving a second, structurally related LQR system (Amos et al., 2018). This results in a nested learning process (Behrouz et al., 2025b): an inner loop that solves the control problem conditioned on the current context, and an outer loop that updates the world-modeling parameters to improve downstream objectives.

A central challenge in integrating optimal control into large-scale language models is computational efficiency. Classical Riccati-based solvers (Bellman, 1954; Kalman et al., 1960) require sequential backward passes and repeated matrix inversions, making them poorly aligned with modern accelerators. We address this challenge through a hardware–algorithm co-design. Leveraging the symplectic structure of LQR dynamics (Bittanti et al., 1991; Laub, 2003), we reformulate the solver by replacing sequential linear-system solves with recursive matrix products and parallel matrix inversions, and further reduce the number of matrix inversions to a constant using structured parameterization. We further fuse this solver into a numerically stable CUDA kernel that efficiently supports both forward planning and backward differentiation.

Building on these components, we propose a hybrid architecture, *TTC-Net*, which interleaves TTC layers with memory-based modules such as attention. Moreover, TTC layer can be inserted as an adapter into pretrained LLMs without modifying the pre-trained base architecture. Empirically, TTC-Net consistently outperforms purely memory-based models on challenging reasoning tasks, including symbolic and mathematical problem solving. We demonstrate that TTC-Net achieves consistent gains on Sudoku games when trained from scratch and improves mathematical reasoning benchmarks up to +27.8% on MATH-500 and $2\text{-}3\times$ Pass@8 improvements on AMC and AIME when inserted and fine-tuned as an adapter to base LLMs.

More broadly, this work proposes a unified view of training and reasoning in language models. Instead of treating supervised learning, RL, and test-time reasoning as separate stages, we internalize goal-directed reasoning as a hardware-efficient optimal control layer within the model architecture – integrating test-time memorization, world modeling, model-based RL, and planning in a single architectural framework, enabling models to reason over future trajectories at inference time while maintaining RL as a persistent objective throughout all training stages (Hatamizadeh et al., 2025; Dong et al., 2025).

2. Background: Memory-Based Architectures

Given a sequence of t tokens $s_1, \dots, s_t$, auto-regressive LLMs learn to predict the next token according to the conditional distribution $p_{\text{LLM}}(s_{t+1}|s_1, \dots, s_t)$. While a variety of architectures have been proposed to model this conditional probability, many of them are derived from associative memory mechanisms (Hopfield, 1982; 1984; Hopfield & Tank, 1985; Schmidhuber, 1992; Ramsauer et al., 2020). In this view, patterns associated with the context $s_1, \dots, s_t$ are stored in memory, and s_{t+1} is synthesized by retrieving information from the contextual representation through the minimization of an energy function.

To be more specific, let $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top \in \mathbb{R}^{n \times d}$ denote a sequence of token embeddings, and let $\mathbf{X}_{\leq t} = [\mathbf{x}_1, \dots, \mathbf{x}_t]^\top$ denote the prefix up to time t . A memory unit is a sequence-to-sequence mapping, whose forward pass can be formulated as learning an online predictor $M_t: \mathbb{R}^d \rightarrow \mathbb{R}$ that encodes the patterns contained in $\mathbf{X}_{\leq t}$ and retrieves relevant information for a given query as the output. The memory is updated online by optimizing a self-supervised regression objective that stores predictive features from the observed prefix:

$$M_t = \arg \min_M \frac{1}{2} \sum_{\tau=1}^t w_{t,\tau} \|M(\mathbf{k}_\tau) - v_\tau\|_2^2 + R(M), \quad (1)$$

where $\mathbf{k}_\tau = \mathbf{W}_K \mathbf{x}_\tau$ and $v_\tau = \mathbf{W}_V \mathbf{x}_\tau$ are two views created from the token $\mathbf{x}_\tau$ through projection matrices $\mathbf{W}_K \in \mathbb{R}^{d \times d}$ and $\mathbf{W}_V \in \mathbb{R}^{1 \times d}$, $w_{t,\tau} \in \mathbb{R}$ are coefficients re-weighting the past tokens, and $R(\cdot)$ is a regularizer over M_t . After obtaining the optimal predictor M_t , a query vector is formed as $\mathbf{q}_t = \mathbf{W}_Q \mathbf{x}_t$ with projection $\mathbf{W}_Q \in \mathbb{R}^{d \times d}$, and the memory unit produces $M_t(\mathbf{q}_t)$ as the output at the t -th position¹. Because models built around this memory module perform an implicit regression over the historical context during the forward pass, they are often considered as *Test-Time Training (TTT)* architectures that can mitigate domain shift and prolong learning at inference time (Liu et al., 2024; Sun et al., 2024; Behrouz et al., 2024; Tandon et al., 2025). Different neural architectures can be interpreted as different instantiations of this paradigm:

Attention. As one of the most popular neural components, attention (Vaswani et al., 2017) has been recognized as a memory mechanism as well (Ramsauer et al., 2020). Under our formulation, attention can be viewed as a non-parametric regressor minimizing Eq. (1) with a mean squared error via the online Nadaraya-Watson estimator (Nadaraya, 1964; Sun et al., 2024). In particular, we define $w_{t,\tau} = 1$ for every t, τ , and $R(M) \equiv 0$ in Eq. (1). Then attention can be

¹Note that it is easy to extend M_t to be a vector-to-vector mapping (e.g., vector-valued value entries in attention) by considering multiple M_t functions solving Eq. (1) with shared key and query vectors.

formulated via the following solution to Eq. (1):

$$M_t(\mathbf{q}) = \sum_{\tau=1}^t \frac{\exp(\mathbf{q}^\top \mathbf{k}_\tau / \sqrt{d}) v_\tau}{\sum_{\tau'=1}^t \exp(\mathbf{q}^\top \mathbf{k}_{\tau'} / \sqrt{d})}, \quad (2)$$

where scaled exponential function $\exp(\mathbf{q}^\top \mathbf{k} / \sqrt{d})$ is chosen as the kernel function. Grounded in non-parametric regression, attention has to traverse through all past tokens (KV caches) before making predictions for the next token.

Linear RNNs. Moving beyond attention, linear RNNs have been proposed as an alternative to avoid the notorious quadratic computational complexity of attention (Gu & Dao, 2023; Sun et al., 2023; Yang et al., 2023b; 2024a; Beck et al., 2024). In test-time training perspective, linear RNNs correspond to parametric solvers of Eq. (1), where M_t becomes a linear mapping parameterized by $\mathbf{h}_t \in \mathbb{R}^d$, such that $M_t(\mathbf{q}) = \mathbf{h}_t^\top \mathbf{q}$. In linear RNNs, $\mathbf{h}_t$ is referred to as the *state*, which tracks all past tokens within a fixed-size memory block. By setting $w_{t,\tau} = 1$ only for $t = \tau$, leveraging gradient descent to update $\mathbf{h}_t$, and flexibly configuring $R(\cdot)$ for Eq. (1), a variety of linear RNNs or SSMS naturally emerge under the following general formulation (Gu & Dao, 2023; Sun et al., 2023; Yang et al., 2023b):

$$\mathbf{h}_t = \mathbf{A}_t \mathbf{h}_{t-1} + \mathbf{B}_t \mathbf{x}_t, \quad (3)$$

where $\mathbf{A}_t \in \mathbb{R}^{d \times d}$ controls what information will be passed to the next state, while $\mathbf{B}_t \in \mathbb{R}^{d \times d}$ encodes the raw inputs to the state space. Different linear RNNs are designed with distinct structures of $\mathbf{A}_t$ and $\mathbf{B}_t$. For instance, when $R(M)$ is removed from the objective Eq. (1) and gradient descent is applied with learning rate η_t , we obtain a new recurrent rule for $\mathbf{h}_t$ with $\mathbf{A}_t = (\mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top)$, $\mathbf{B}_t = \eta_t \mathbf{k}_t \mathbf{W}_V^\top$. This corresponds to DeltaNet, as proposed in Schlag et al. (2021); Peng et al. (2025a); Yang et al. (2024b)².

3. A Planning-Based Neural Architecture

3.1. Learning to Reinforce Learning at Test Time

Our key idea is to model a layer for next-token prediction as the optimal decision induced by a tractable Markov Decision Process (MDP) environment, thereby programming a goal-seeking objective into the LLM architecture. Suppose we are predicting the $(\tau + 1)$ -th token given the prior context embeddings $[\mathbf{x}_1, \dots, \mathbf{x}_\tau]$, we begin by denoting an initial state as $\mathbf{h}_0 \in \mathbb{R}^d$, which encodes the context $[\mathbf{x}_1, \dots, \mathbf{x}_\tau]$ up to time τ . Given a planning horizon $T > 0$, we denote the latent state and token prediction at t steps ahead as $\mathbf{h}_t, \mathbf{u}_t \in \mathbb{R}^d$, respectively. The goal is to output the first-step prediction that achieves optimality of the following

²Similarly, by considering multiple parallel M_t 's, $\mathbf{h}_t$ can be extended to matrix-valued states as in Dao & Gu (2024); Yang et al. (2023b).

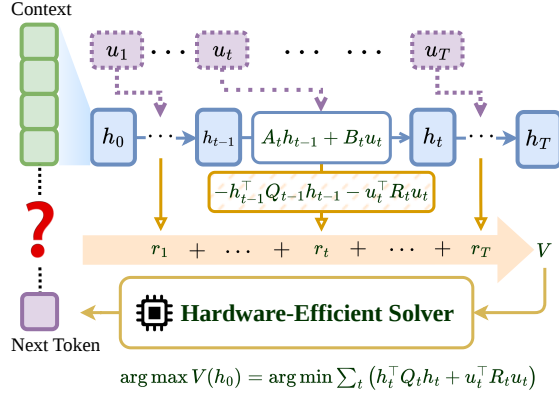


Figure 2. **Test-Time Control (TTC) layer.** To predict the next token, TTC layers think through a receding-horizon LQR problem with linear state evolution and a cost function. A *hardware-efficient solver* computes optimal actions using structured matrix operations, enabling test-time control with minimal inference overhead.

Bellman equation: $\mathbf{u}_1^* = \arg \max_{\mathbf{u}} Q_0(\mathbf{h}_0, \mathbf{u})$:

$$Q_t(\mathbf{h}_t, \mathbf{u}) = r_t(\mathbf{h}_t, \mathbf{u}) + V_{t+1}(f_{t+1}(\mathbf{h}_t, \mathbf{u})), \quad (4)$$

$$V_t(\mathbf{h}_t) = \max_{\mathbf{u}} Q_t(\mathbf{h}_t, \mathbf{u}), \quad (5)$$

where $0 \leq t \leq T-1$, $f_t : (\mathbf{h}_{t-1}, \mathbf{u}_t) \mapsto \mathbf{h}_t$ models the state transition, $r_t(\mathbf{h}_t, \mathbf{u}_{t+1})$ is the reward function, Q_t and V_t are known as Q-function and value function, respectively. We let $V_T(\mathbf{h}_T) = r_T(\mathbf{h}_T)$ as the boundary condition. Solving a general Bellman equation as a neural network layer is computationally infeasible.

Instead, we can consider a more tractable yet expressive family of Eq. (4). Specifically, we adopt a linear dynamical system to model the state transition and optimize a quadratic reward function:

$$f_t(\mathbf{h}_{t-1}, \mathbf{u}_t) = \mathbf{A}_t \mathbf{h}_{t-1} + \mathbf{B}_t \mathbf{u}_t, \quad (6)$$

$$r_t(\mathbf{h}_t, \mathbf{u}_{t+1}) = -(\mathbf{h}_t^\top \mathbf{Q}_t \mathbf{h}_t + \mathbf{u}_{t+1}^\top \mathbf{R}_{t+1} \mathbf{u}_{t+1}), \quad (7)$$

where $\{(\mathbf{A}_t \in \mathbb{R}^{d \times d}, \mathbf{B}_t \in \mathbb{R}^{d \times d})\}_{1 \leq t \leq T}$ are matrices that govern the evolution of the state induced by the upcoming token $\mathbf{u}_t$, and $\{(\mathbf{Q}_t \in \mathbb{R}^{d \times d}, \mathbf{R}_t \in \mathbb{R}^{d \times d})\}_{1 \leq t \leq T}$ are symmetrical positive semi-definite matrices defining quadratic cost functions over the state $\mathbf{h}_t$ and action token $\mathbf{u}_t$. In particular, $\mathbf{Q}_0 = \mathbf{0}$ and $r_T(\mathbf{h}_T) = -\mathbf{h}_T^\top \mathbf{Q}_T \mathbf{h}_T$ for the boundary cases. Despite the linearity of the state transition function, the linear dynamical systems (Eq. (6)) have been shown to be sufficiently powerful to represent a broad family of MDPs (Hafner et al., 2019; Gu et al., 2020; 2021b;a). The objective in Eq. (8) naturally accommodates both process rewards $r_t(\mathbf{h}_t, \mathbf{u}_{t+1})$ through $\{(\mathbf{Q}_t, \mathbf{R}_t)\}_{1 \leq t \leq T-1}$ and an outcome reward $r_T(\mathbf{h}_T)$ via $\mathbf{Q}_T$ at the terminal step. Fig. 2 illustrates the design of our layer.

In fact, the MDP system in Eq. 4 is equivalent to solving a constrained optimization over the unrolled states and

forecast tokens for decisions $\mathbf{u}_1, \dots, \mathbf{u}_T$:

$$\min_{\mathbf{u}_1, \dots, \mathbf{u}_T} \frac{1}{2} \sum_{t=1}^T (\mathbf{h}_t^\top \mathbf{Q}_t \mathbf{h}_t + \mathbf{u}_t^\top \mathbf{R}_t \mathbf{u}_t) \quad (8)$$

$$\text{s.t. } \mathbf{h}_t = \mathbf{A}_t \mathbf{h}_{t-1} + \mathbf{B}_t \mathbf{u}_t, \quad 1 \leq t \leq T.$$

Among all $\mathbf{u}_1^*, \dots, \mathbf{u}_T^*$, we take the first-step optimal decision $\mathbf{u}_1^*$ as the output for the next token representation. Eq. (8) is the well-studied receding-horizon *Linear-Quadratic Regulator (LQR)* (Bellman, 1954; Kalman et al., 1960). The solution to an LQR problem in Eq. (8) is known as:

Proposition 3.1 (Riccati Iteration). *The optimal $\mathbf{u}_1^*$ minimizing Eq. (8) depends linearly on $\mathbf{h}_0$ as $\mathbf{u}_1^* = \mathbf{K}_1^* \mathbf{h}_0$, where:*

$$\mathbf{K}_t^* = -(\mathbf{R}_t + \mathbf{B}_t^\top \mathbf{P}_t \mathbf{B}_t)^{-1} \mathbf{B}_t^\top \mathbf{P}_t \mathbf{A}_t, \quad (9)$$

$$\mathbf{P}_t = \mathbf{Q}_t + \mathbf{A}_{t+1}^\top \mathbf{P}_{t+1} \mathbf{A}_{t+1} - (\mathbf{B}_{t+1}^\top \mathbf{P}_{t+1} \mathbf{A}_{t+1})^\top (\mathbf{R}_{t+1} + \mathbf{B}_{t+1}^\top \mathbf{P}_{t+1} \mathbf{B}_{t+1})^{-1} (\mathbf{B}_{t+1}^\top \mathbf{P}_{t+1} \mathbf{A}_{t+1}), \quad (10)$$

for $t = 1, \dots, T$, and $\mathbf{P}_T = \mathbf{Q}_T$.

In addition, we can show that value function $V_t(\mathbf{h}_t) = -\frac{1}{2} \mathbf{h}_t^\top \mathbf{P}_t \mathbf{h}_t$ (see Appendix A.1). Thus, $\mathbf{P}_t$ is often called a value matrix. Eq. (10) computing the value matrices $\{\mathbf{P}_t\}_{1 \leq t \leq T}$ is also known as the Riccati iteration (Bellman, 1954; Kalman et al., 1960). It involves a backward iteration from time T to 1 to compute the value of $\mathbf{P}_1$ and $\mathbf{K}_1$. This implies that we are not able to solve $\mathbf{u}_1^*$ by only considering past or local information at time $t = 1$ because $\mathbf{u}_1^*$ has long-term influence over all subsequent states.

Since the forward pass of our proposed module decodes a memory state $\mathbf{h}_0$ into the optimal decision $\mathbf{u}_1^*$ by internally solving an optimal control problem, we refer to our proposed module as a *Test-Time Control (TTC)* layer. TTC layer is parameterized via $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T$, and we denote it end-to-end as $\text{TTC}(\mathbf{h}_0, \{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T)$. Unlike a memory-based layer that minimizes regression objectives over historical data points, a TTC layer optimizes for future trajectories and minimizes costs incurred in the long shot. Solving for the optimal action $\mathbf{u}_t^*$ during the model forward requires computing the value function on the fly at inference time. This process endows each sequential modeling block with an internal value function V_t and a nested model-based RL objective.

3.2. Differentiating TTC Layers

To train a TTC layer end-to-end, suppose we obtain the output $\mathbf{o} = \text{TTC}(\mathbf{h}_0, \{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T)$ and compute loss $\ell(\mathbf{o})$ in forward pass. Then, during the backward pass, we receive gradients $\nabla_{\mathbf{o}} \ell$ backpropagated from the overall loss function. Our goal is to leverage this gradient to compute gradients with respect to the TTC parameters $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T$.

To show how this can be achieved, we need to demonstrate that the TTC objective (Eq. (8)) can be cast into a KKT form: $\mathcal{L} = \frac{1}{2} \sum_{t=1}^T (\mathbf{h}_t^\top \mathbf{Q}_t \mathbf{h}_t + \mathbf{u}_t^\top \mathbf{R}_t \mathbf{u}_t) + \sum_{t=1}^T \lambda_t^\top (\mathbf{A}_t \mathbf{h}_{t-1} + \mathbf{B}_t \mathbf{u}_t - \mathbf{h}_t)$ where $\lambda_t \in \mathbb{R}^d$ are defined as Lagrangian multipliers, also referred to as *co-states* in LQR. A standard approach to solving the KKT system for $\{(\mathbf{h}_t^*, \mathbf{u}_t^*, \lambda_t^*)\}_{t=1}^T$ is to first compute the optimal control $\mathbf{u}_1^*$ via Riccati iteration (Proposition 3.1), and then alternatively compute the new state $\mathbf{h}_t$ via Eq. (6) and the next optimal control via $\mathbf{u}_{t+1} = \mathbf{K}_{t+1} \mathbf{h}_t$ where $\mathbf{K}_{t+1}$ is as defined in Eq. (9). Afterwards, another backward scanning is performed to obtain co-states through this relation: $\lambda_t = \mathbf{A}_{t+1}^\top \lambda_{t+1} + \mathbf{Q}_t \mathbf{h}_t$, which is derived from the KKT condition $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \mathbf{0}$. See Appendix A.2 for the derivation.

With these notions, we can represent gradients w.r.t. $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T$ and $\mathbf{h}_0$ using states, actions, and co-states via the approach introduced in Amos & Kolter (2017); Amos et al. (2018).

Theorem 3.2. Consider a TTC layer with parameters $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T$ and initial state $\mathbf{h}_0$, suppose we have output $\mathbf{o} = \text{TTC}(\mathbf{h}_0, \{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T)$, the loss is computed as $\ell(\mathbf{o})$, and we have obtained $\nabla_{\mathbf{o}} \ell$. Then the gradients w.r.t. $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}$ are:

$$\begin{aligned} \nabla_{\mathbf{A}_t} \ell &= \lambda_t^* \tilde{\mathbf{h}}_{t-1}^{*\top} + \tilde{\lambda}_t^* \mathbf{h}_{t-1}^{*\top}, & \nabla_{\mathbf{B}_t} \ell &= \lambda_t^* \tilde{\mathbf{u}}_t^{*\top} + \tilde{\lambda}_t^* \mathbf{u}_t^{*\top}, \\ \nabla_{\mathbf{Q}_t} \ell &= \frac{1}{2} (\tilde{\mathbf{h}}_t^* \mathbf{h}_t^{*\top} + \mathbf{h}_t^* \tilde{\mathbf{h}}_t^{*\top}), & \nabla_{\mathbf{R}_t} \ell &= \frac{1}{2} (\tilde{\mathbf{u}}_t^* \mathbf{u}_t^{*\top} + \mathbf{u}_t^* \tilde{\mathbf{u}}_t^{*\top}), \end{aligned}$$

and gradient w.r.t. $\mathbf{h}_0$ can be computed by $\nabla_{\mathbf{h}_0} \ell = \tilde{\lambda}_0^*$, where $\{(\mathbf{h}_t^*, \lambda_t^*, \mathbf{u}_t^*)\}_{t=1}^T$ are the solution to the KKT system associated with Eq. (8) and $\{(\tilde{\mathbf{h}}_t^*, \tilde{\lambda}_t^*, \tilde{\mathbf{u}}_t^*)\}_{t=1}^T$ are the solution to the KKT system corresponding to the following LQR system with $\tilde{\mathbf{h}}_0 = \mathbf{0}$:

$$\begin{aligned} \min \frac{1}{2} \sum_{t=1}^T (\tilde{\mathbf{h}}_t^\top \mathbf{Q}_t \tilde{\mathbf{h}}_t + \tilde{\mathbf{u}}_t^\top \mathbf{R}_t \tilde{\mathbf{u}}_t) + \nabla_{\mathbf{o}} \ell^\top \tilde{\mathbf{u}}_1 \quad (11) \\ \text{s.t. } \tilde{\mathbf{h}}_t = \mathbf{A}_t \tilde{\mathbf{h}}_{t-1} + \mathbf{B}_t \tilde{\mathbf{u}}_t, \quad 1 \leq t \leq T. \end{aligned}$$

We refer to the LQR solved in the forward pass as the *primal LQR*, and the additional LQR solved during the backward pass as the *dual LQR*. Compared to the primal LQR, the dual LQR assumes zero initial state while introducing an additional affine term $\nabla_{\mathbf{o}} \ell^\top \tilde{\mathbf{u}}_1$ in the cost function. Despite these differences, solving the primal and dual LQRs shares the same underlying algorithm (see Appendix A for a unified derivation to account for the affine term). The gradients with respect to the primal LQR parameters are then obtained by combining the KKT solutions from both the primal and dual LQRs, using Kronecker product.

3.3. Hardware Co-Design for TTC

As seen in Sec. 3.1 and 3.2, one inference iteration of TTC requires solving one LQR while one training iteration re-

quires solving two LQRs. In practice, incorporating TTC layers into LLMs requires both training and inference to operate over thousands of tokens and long planning horizons. This setting makes conventional solvers based on Riccati iteration (Proposition 3.1) (Amos et al., 2018; Hansen et al., 2023) difficult to apply. We observe that Riccati iteration is a simultaneously compute- and I/O-bound algorithm. (1) Riccati iteration computes $\mathbf{P}_t$ sequentially, requiring a matrix inversion at each time step. This leads to a computational complexity of $O(Td^3)$. Such inversions are poorly supported by dedicated hardware accelerators and cannot be parallelized across the horizon due to inherent sequential dependencies. (2) Each Riccati iteration also involves a sequence of matrix multiplications. These operations incur substantial memory traffic between GPU high-bandwidth memory (HBM) and on-chip SRAM, resulting in significant I/O overhead. To address these challenges, we propose a new solver that improves parallelizability and hardware utility for solving LQR problems.

Hardware-Efficient LQR Solver. Our new algorithm solving LQR is shown by the following result:

Theorem 3.3 (Symplectic Iteration). The first-step solution to $\min \sum_{t=1}^T \left[\frac{1}{2} (\mathbf{h}_t^\top \mathbf{Q}_t \mathbf{h}_t + \mathbf{u}_t^\top \mathbf{R}_t \mathbf{u}_t) + \mathbf{r}_t^\top \mathbf{u}_t \right]$ s.t. $\mathbf{h}_t = \mathbf{A}_t \mathbf{h}_{t-1} + \mathbf{B}_t \mathbf{u}_t$ can be computed by $\mathbf{u}_1^* = -\mathbf{R}_1^{-1} (\mathbf{B}_1^\top (\mathbf{A}_1^\top)^{-1} \mathbf{Y}_1^{-1} \mathbf{Y}_2 (\mathbf{h}_0 + \mathbf{y}_3) + \mathbf{r}_1)$ such that:

$$[\mathbf{Y}_1 \quad \mathbf{Y}_2] = [\mathbf{I} \quad \mathbf{Q}_T] \prod_{t=1}^T \Sigma_{T-t+1}, \quad (12)$$

$$\mathbf{y}_3 = [\mathbf{I} \quad \mathbf{Q}_T] \sum_{t=1}^T \left(\prod_{r=1}^{T-t} \Sigma_{T-r+1} \right) \begin{bmatrix} \mathbf{0} \\ -\mathbf{B}_t \mathbf{R}_t^{-1} \mathbf{r}_t \end{bmatrix},$$

$$\Sigma_t = \begin{bmatrix} (\mathbf{A}_t^\top)^{-1} & (\mathbf{A}_t^\top)^{-1} \mathbf{Q}_{t-1} \\ \mathbf{G}_t (\mathbf{A}_t^\top)^{-1} & \mathbf{A}_t + \mathbf{G}_t (\mathbf{A}_t^\top)^{-1} \mathbf{Q}_{t-1} \end{bmatrix}, \quad (13)$$

$$\mathbf{G}_t = \mathbf{B}_t \mathbf{R}_t^{-1} \mathbf{B}_t^\top. \quad (14)$$

Theorem 3.3 yields a solver for both the primal and dual LQRs. Note that Σ_t is a symplectic matrix, and computing the first-step action $\mathbf{u}_1^*$ requires a reverse iterative matrix product from $t = T$ to $t = 1$ (see Eq. (12)). Such an iterative matrix product is called (*reverse*) *symplectic iteration*. A key observation from Theorem 3.3 is that it replaces the sequential Riccati recursion dominated by dense matrix inversions with a cumulative matrix product over Σ_t , utilizing the symplectic structure of Σ_t . Importantly, the matrix inverses appearing within each Σ_t are independent across time steps and can therefore be computed fully in parallel. Outside of this cumulative product, only a single dense matrix inversion is required to recover the optimal first-step decision $\mathbf{u}_1^*$. By Theorem 3.3, all remaining sequential computation is confined to matrix multiplication, which enables effective utilization of dedicated hardware

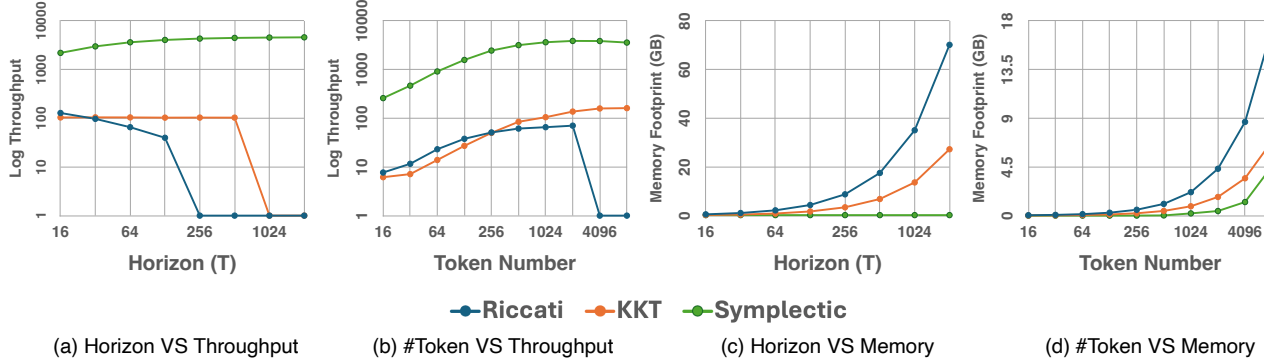


Figure 3. **Benchmarking running speed and memory of different LQR solvers.** Throughput is reported in TFLOPs/s on a logarithmic scale. Among all evaluated solvers, our method achieves over a **10x** higher throughput while maintaining constant memory cost w.r.t. horizon. Zero throughput indicates an out-of-memory error during execution.

accelerators (e.g., Tensor Cores) for high-throughput dense linear algebra. The algorithm implied from Theorem 3.3 to compute u_1^* is listed in Algorithm 3.

As seen in Theorem 3.2, backpropagation through TTC layers requires access to the full trajectories of both the primal and dual LQRs. As we will show in Theorem A.4, Appendix A, there exists a *forward symplectic iteration* involving the symplectic adjoint of Σ_t in Eq. (12) that computes each (h_t, u_t, λ_t) via an iterative matrix-vector product from $t = 1$ to $t = T$. This result provides a unified implementation for computing u_1^* and performing rollouts.

Structured Parameterization. In fact, we can further specialize to a structured family of LQRs in which $\{A_t\}_{t=1}^T$ and $\{R_t\}_{t=1}^T$ are defined to be diagonal matrices. This structure makes all inversion operations on A_t and R_t significantly more efficient, reducing the number of nontrivial dense matrix inversions from $O(T)$ to $O(1)$. We will adopt this efficient parameterization in our implementation of the TTC layer. As demonstrated in prior work (Gupta et al., 2022; Gu et al., 2022; Gu & Dao, 2023), diagonalized SSMs can be as expressive as dense SSMs. In contrast, diagonalization of A_t cannot simplify the computation for Riccati iteration as P_t remains a dense matrix in Eq. (10).

Kernel Optimization. The primary computation in the symplectic iteration algorithm (Theorem 3.3), which relies on sequential matrix multiplications, is well suited for kernel fusion. Accordingly, we fuse Eq. 12 into a CUDA kernel to reduce memory I/O traffic and further improve computational efficiency. We also optimize numerical stability and the backward pass. See details in Appendix B.

Empirical Validation. We benchmark the training throughput and memory footprint of our fused symplectic iteration solver and compare it against two classic baselines: (1) a native Riccati iteration solver with gradients computed via automatic differentiation; and (2) a KKT-based solver

(Amos et al., 2018) replacing gradient computation with a manner similar to Theorem 3.2. As shown in Fig. 3, the symplectic iteration solver achieves over a $10\times$ higher throughput than both the Riccati- and KKT-based solvers. Meanwhile, the two classic solvers are bottlenecked by memory overhead, whereas our solver scales much more efficiently with the planning horizon and the number of tokens to be processed.

4. TTC-Net: A Hybrid Model with TTC Layer

In this section, we introduce the key techniques that turn TTC layers into a language model.

Contextualization. Learning a single set of LQR parameters $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$ shared across all test samples enforces a fixed decision pattern, which limits adaptability to specific contexts and disables flexible adjustment of the planning horizon. Meanwhile, constraining $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$ to be uniform across the horizon further restricts expressivity, hindering the model’s ability to capture temporal variations in the environment dynamics and cost functions (including discounting effects) (Nhu et al., 2025). Combining these observations, we propose to condition the TTC parameters on both the initial state h_0 , which encodes the context, and the time step t . Given an initial state h_0 , we first generate a group of time modulation coefficients $\Gamma_{\square} = \exp(-\text{diag}(s_{\Gamma_{\square}}(h_0)))$ where $\square \in \{A, B, Q\}$. Then we synthesize LQR parameters by modulating power series of $\Gamma_{\square}$:

$$A_t = I + \Gamma_A^t \text{diag}(s_A(h_0)), \quad B_t = \sum_{i=1}^r s_B(h_0)_i B^{(i)} \Gamma_B^t, \\ Q_t = \Gamma_Q^t \left(\sum_{i=1}^r s_Q(h_0)_i Q^{(i)} \right) \Gamma_Q^t, \quad R_t = \text{diag}(s_R(h_0)),$$

where we single out the terminal cost parameter Q_T and override it with $Q_T = \sum_{i=1}^r s_{Q_f}(h_0)_i Q^{(i)}$. $s_{\square}$ denotes a linear layer with task-specific activations for every $\square \in$

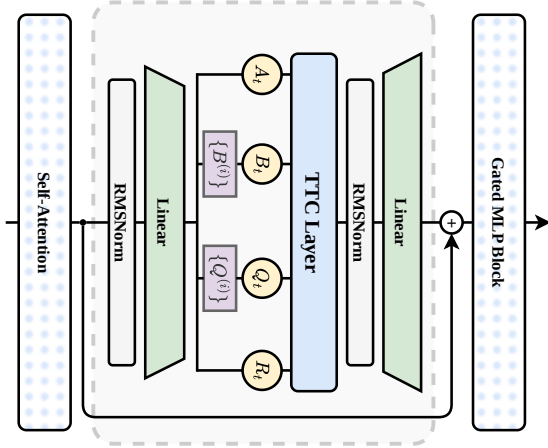


Figure 4. **Overview of TTC-Net.** We construct a hybrid model by inserting a TTC layer between attention and MLP.

$\{A, B, Q, R, Q_f, \Gamma_A, \Gamma_B, \Gamma_Q\}$. In particular, $s_{\Gamma_\square}$, s_R , and s_Q employ a softplus activation, necessary for Q_t and R_t 's positive semi-definiteness. A_t is parameterized with an added identity term to ensure invertibility in Σ_t . $\{Q^{(i)}\}_{i=1}^r$ and $\{B^{(i)}\}_{i=1}^r$ form two groups of basis matrices, which are linearly combined using context-informed coefficients. Every $Q^{(i)} = Q_c^{(i)} Q_c^{(i)\top} / \sqrt{d}$ adopts a Cholesky reparameterization with unconstrained $\{Q_c^{(i)}\}_{i=1}^r$. All $\Gamma_\square$ take values in $(0, 1)$, thereby discounting the effect of the state unrolled over the horizon. In practice, time-dependent parameters are not instantiated in HBM; instead they are computed on the fly within the kernel.

Hybrid with Memory Layers. The TTC layer operates by taking an expressive memory state as input. Consequently, TTC layers need to be interleaved with memory-based modules and form a hybrid architecture for language modeling. In practice, we insert a TTC layer between attention and MLP every 8 transformer blocks. We refer to the resulting architecture as *TTC-Net*. In TTC-Net, a TTC layer is applied independently to each token. TTC-Net employs a linear mapping to project context-rich token features produced by attention into the initial state of the TTC layer. The output of the TTC layer is then normalized and passed through another linear mapping W_{out} , after which it is added back to the residual stream of the LLM backbone. Please refer to Fig. 4 for a quick overview and Appendix C for the formulation. TTC-Net can be trained either from scratch or by continuing training from a pretrained model. When fine-tuning from a pretrained model, we initialize the output projection $W_{out} = 0$ so that the initial model remains identical to the original backbone.

More Architecture Designs. In addition to the core ingredients above, we defer our design of multi-head structures, parameterizations of contextualization modules, training strategies, and test-time scaling techniques to Appendix C.

Table 1. Accuracy (%) on Sudoku reasoning. Best method (ours) is marked in **bold** while the runner-up is underlined.

Methods	Single-Step		Multi-Step	
	Board Acc.	Cell Acc.	Board Acc.	Cell Acc.
Transformer	<u>58.50</u>	86.54	90.10	94.08
Mamba	54.60	85.50	88.60	91.29
Mamba2	55.50	85.10	87.20	90.52
GDN	57.30	87.19	89.80	93.70
Samba	57.20	<u>87.99</u>	<u>90.40</u>	<u>94.61</u>
TTC-Net (Ours)	61.30	90.17	93.40	97.33

5. Experiments

5.1. Sudoku Solving

Sudoku is a classical logical puzzle that requires long-horizon reasoning, constraint propagation, and multi-step planning to reach a valid solution, making it a canonical benchmark for evaluating structured reasoning and planning capabilities of deep learning models (Palm et al., 2018; Du et al., 2019; Wang et al., 2025a; Long, 2023). We leverage Sudoku as a controlled synthetic task on long-horizon reasoning to evaluate the effectiveness of TTC-Net.

Data. We adopt the 10k 9×9 boards from a challenging Sudoku dataset introduced by Palm et al. (2018), where each board contains between 17 and 34 given digits. We reserve 1k boards for evaluation and use the remaining 9k boards for training. Each Sudoku board is tokenized as a sequence, with each cell represented by a token from the vocabulary $\{[\text{mask}], 1, \dots, 9\}$. These cell tokens are mapped to token embeddings, and the resulting sequence is passed into a sequential processing backbone.

Baselines. We compare TTC-Net against a range of popular neural architectures for sequential modeling: (1) a Transformer with pure attention blocks (Vaswani et al., 2017), (2) Mamba and Mamba2 (Gu & Dao, 2023; Dao & Gu, 2024), (3) GDN (Yang et al., 2024a), and (4) Samba (Ren et al., 2024). We adopt positional embeddings and full self-attention for transformer blocks. We include GDN as it has been shown to excel at state tracking (Grazzi et al., 2024; Peng et al., 2025a). Samba represents a hybrid architecture that combines the Mamba layer with sliding-window attention. All SSM, linear attention, and sliding-window attention layers scan the sequence twice bidirectionally to propagate information across the Sudoku grid. Despite their architectural differences, all of these baselines fall under the category of memory-based approaches. All models consist of 32 layers in total. A classifier is attached to the backbone to decode the representations into digit predictions.

Training and Evaluation. During training, we apply a cross-entropy loss over all masked cells. In addition, we decode the output of each block through the classifier and ap-

Table 2. **Accuracy (%) on math reasoning datasets** compared with baseline methods, including test-time memorization approaches. We mark the best performer in **bold** and the runner-up with underline. Our TTC-Net achieves the highest level of accuracy consistently.

Models	MATH-500	AMC		AIME24		AIME25	
		Acc@8	Pass@8	Acc@8	Pass@8	Acc@8	Pass@8
Base model	25.00	6.63	31.32	0.00	0.00	0.00	0.00
Full Finetuning	46.80	<u>20.78</u>	<u>46.98</u>	1.67	6.67	0.00	0.00
+ MLP (Houlsby et al., 2019)	40.80	18.67	33.73	0.00	0.00	0.00	0.00
+ Attention (Vaswani et al., 2017)	47.00	20.48	44.58	0.42	3.33	1.25	<u>6.67</u>
+ RetNet (Sun et al., 2023)	42.60	19.58	39.76	<u>2.50</u>	<u>13.33</u>	0.00	0.00
+ Mamba (Gu & Dao, 2023)	44.80	22.29	44.58	0.83	3.33	<u>1.67</u>	3.33
+ GDN (Yang et al., 2024a)	<u>47.80</u>	17.77	37.35	0.42	3.33	0.83	<u>6.67</u>
+ MesaNet (von Oswald et al., 2025)	47.40	12.65	27.71	1.25	10.00	0.00	0.00
TTC-Net	52.80	23.34	54.22	3.33	20.00	5.00	20.00

ply the same loss to supervise intermediate representations, encouraging the model to make progressive corrections at every layer, following Yang et al. (2023c). We list all training configurations in Appendix E.7. We evaluate the models using two approaches: (1) performing a single forward pass and measuring accuracy on the masked cells; and (2) iteratively completing one cell at a time by selecting the most confident prediction at each step, repeating this process until all cells are filled, akin to CoT (Wei et al., 2022) (see Algorithm 6). For both approaches, we report both cell- and board-level accuracy. Throughout training and testing, we fix $T_{train} = T_{test} = 4$ for TTC-Net.

Results. Tab. 1 reports the performance of all evaluated architectures. TTC-Net outperforms all baselines by a clear margin in accuracy. In particular, TTC-Net surpasses the strongest runner-up baseline, i.e., Transformer, by 2.8%, in board-level accuracy on single-step completion. Moreover, TTC-Net demonstrates highly coherent multi-step reasoning, as evidenced by its superior accuracy on multi-step Sudoku solving. In fact, this performance gain is expected, as TTC-Net is explicitly designed to internalize a long-horizon reasoning mechanism that is well-suited for solving Sudoku. Sudoku can be viewed as a constraint satisfaction problem (Wang et al., 2019; Yang et al., 2023c). The TTC objective (Eq. (8)) provides an effective approximation by casting Sudoku solving as a sequential decision-making process, where placing a digit on the board corresponds to a linear state transition, and the quadratic cost function serves as a smooth surrogate for constraint satisfaction barriers.

5.2. Math Reasoning

Settings. In this section, we evaluate the effectiveness of TTC-Net on mathematical reasoning tasks for LLM. Rather than training models from scratch, we adopt a continual learning setting in which we perform supervised fine-tuning (SFT) on pretrained models augmented with additional architectural modules. From this perspective, SFT can be

viewed as a form of imitation learning, and training the TTC layers corresponds to an instance of inverse RL, where TTC layers learn to infer latent state transition and objectives that facilitate cloning the expert behavior. We use Llama-3-Instruct-7B as the base model and initialize TTC-Net from the base model with zero initialization applied to the output projection (see Sec. 4). To enable a fair comparison, we construct hybrid architectures by inserting an additional memory layer with a comparable number of parameters between the attention and MLP every 8 transformer blocks. All newly introduced layers are trained jointly with the rest of the model. The new layers play a role similar to adapter modules that allocate additional learning capacity for acquiring new reasoning abilities. The memory mechanisms considered in this study include attention (Vaswani et al., 2017), RetNet (Sun et al., 2023), Mamba (Gu & Dao, 2023), GDN (Yang et al., 2024a), and MesaNet (von Oswald et al., 2025).

Benchmark Evaluation. We fine-tune all baselines for one epoch on the OpenThoughts2-114K dataset (Guha et al., 2025), along with an additional 800K self-collected and curated reasoning examples. See Appendix E.7 for more information about the curated dataset and the training hyperparameters. To disentangle improvements due to architectural modifications from those arising solely from fine-tuning, we additionally report the performance of supervised fine-tuning (SFT) applied only to the base model weights, without introducing new modules. We evaluate reasoning performance on a suite of challenging benchmarks, including Math-500 (Hendrycks et al., 2021), AMC (LI et al., 2024), AIME 2024, and AIME 2025. We adopt $T_{test} = 8, 16, 16$ at inference time for Math-500, AMC, and AIME, respectively. For AMC and AIME, we sample 8 responses per prompt using temperature sampling and report both Avg@8 and Pass@8 accuracy following Zuo et al. (2025). The results are summarized in Tab. 2. Across all benchmarks, TTC-Net consistently outperforms

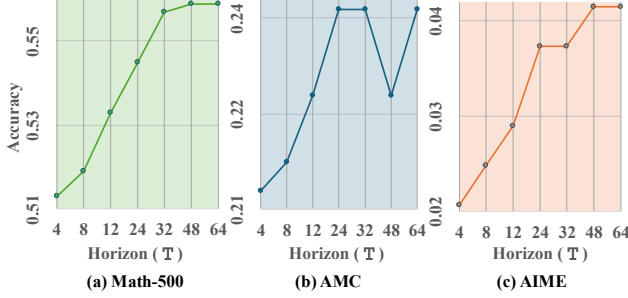


Figure 5. Test-time scaling with TTC layers. TTC allows scaling test-time compute to improve performance by enlarging the planning horizon T .

all other fine-tuned hybrid architectures. Notably, the base model achieves zero accuracy on the challenging AIME 2024 and AIME 2025 datasets, whereas TTC-Net exhibits clear performance emergence, highlighting its ability to unlock complex reasoning capabilities. In contrast, hybrid models augmented with additional memory layers fail to stably surpass SFT applied to the base model alone. Notably, TTC-Net achieves large gains in Pass@8 accuracy, suggesting that the TTC layer extends the effective reasoning boundary of the base model, in contrast to prior observations that post-training alone cannot overcome the intrinsic capability ceiling of the backbone (Yue et al., 2025). These results indicate that the control-based objective underlying the TTC layer provides a critical inductive mechanism for complex reasoning in LLMs.

Test-Time Scaling. As discussed in Sec. 4, the TTC layer supports a flexible planning horizon and can achieve improved performance with longer horizons. Accordingly, we evaluate the fine-tuned TTC-Net by extending its planning horizon at test time to examine how performance scales with T . The results, shown in Fig. 5, validate our hypothesis: reasoning accuracy consistently improves as the planning horizon increases. Notably, although the maximum horizon used during training is 32, the model generalizes successfully to $T = 64$ at test time and continues to enhance the reasoning performance.

5.3. Ablation Studies

We conduct ablation studies to evaluate the main design choices of TTC-Net. The detailed results are deferred to Appendix E.1, while we summarize the findings below. Time-heterogeneous parameterization is important for both accuracy and horizon extrapolation: the time-homogeneous variant achieves 48.40% at $T_{\text{test}} = 8$ and drops to 45.70% at $T_{\text{test}} = 16$, while the full time-heterogeneous model improves to 52.80% and 53.60%, respectively. Horizon sampling is also critical: fixed-horizon training performs reasonably at the matched horizon but degrades sharply from 50.60% to 31.50% when evaluated at a longer horizon,

whereas sampled horizons preserve extrapolation. The interleaving pattern affects the accuracy-cost trade-off. Distributing TTC layers more evenly with attention layers yields stronger performance than sparse or consecutive insertion. These results support our default design using time-heterogeneous control parameters, Poisson log-normal horizon sampling, and an 8:1 attention-to-TTC ratio.

5.4. Additional Evaluations

We conduct several additional experiments: we compare TTC-Net against decoding-based test-time scaling methods (e.g., BoN and ToT), evaluate the fine-tuned models beyond math reasoning tasks, test its effectiveness on a stronger Qwen2.5-Math-7B backbone, analyze inference efficiency under different planning horizons, and analyze TTC latent trajectories. These experiments collectively examine the generality, efficiency, and interpretability of TTC-Net. Please find the full results and detailed analyses in Appendix E.

6. Conclusion

We introduce Test-Time Control (TTC)-Net, a new architectural paradigm that augments memory-based language models with test-time control. At its core, TTC-Net embeds TTC layers that formulate reasoning as an optimal control problem over internal representations, enabling models to plan future trajectories before decoding the next token. To make this paradigm scalable, we derive a hardware-efficient LQR solver that supports parallel execution with minimal overhead, allowing TTC layers to be deployed as lightweight adapters within pretrained LLMs. Empirically, TTC-Net consistently improves reasoning performance. More broadly, TTC-Net reframes test-time learning as structured decision making rather than memorization or parameter adaptation, unifying memory, world modeling, reinforcement learning objectives, and long-horizon planning within a single architecture.

Limitations and Future Works. Theoretically, although a single TTC layer has a well-defined and interpretable optimization objective, it remains unclear how multiple TTC layers jointly interact and represent dynamics within a deep transformer. A rigorous theoretical analysis from this perspective remains an open direction for future research. Empirically, it is worthwhile to explore more expressive parameterizations of the latent-space dynamics and reward modeling, including advanced linear approaches (Yang et al., 2024b) and even non-linear formulations (Amos et al., 2018), while maintaining compatibility with hardware-level optimization constraints. While our results demonstrate the promise of TTC layers as core components of LLMs, a more comprehensive evaluation across larger-scale models and all stages of training remains open for future work.

Acknowledgements

PW thanks Liangzu Peng, Junbo Li, and Zun Wang for insightful discussions and for pointing to useful resources. PW also thanks Ruisi Cai for proofreading this manuscript. This work was done when PW was at Amazon. PW is in part supported by Google PhD Fellowship in Machine Learning and ML Foundations.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential scientific implications with societal benefits by this work, including reduced computation (thereby energy consumption) and lower memory requirement due to our new TTC layer and differentiable formulation, in addition to added capability of reasoning in language models.

References

- Amos, B. and Kolter, J. Z. Optnet: Differentiable optimization as a layer in neural networks. In *International conference on machine learning*, pp. 136–145. PMLR, 2017.
- Amos, B., Jimenez Rodriguez, I. D., Sacks, J., Boots, B., and Kolter, J. Z. Differentiable mpc for end-to-end planning and control. *Advances in neural information processing systems*, 31, 2018.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.
- Behrouz, A., Zhong, P., and Mirrokni, V. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- Behrouz, A., Li, Z., Kacham, P., Daliri, M., Deng, Y., Zhong, P., Razaviyayn, M., and Mirrokni, V. Atlas: Learning to optimally memorize the context at test time. *URL https://arxiv.org/abs/2505.23735*, 2025a.
- Behrouz, A., Razaviyayn, M., Zhong, P., and Mirrokni, V. Nested learning: The illusion of deep learning architectures. *arXiv preprint arXiv:2512.24695*, 2025b.
- Behrouz, A., Razaviyayn, M., Zhong, P., and Mirrokni, V. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization. *arXiv preprint arXiv:2504.13173*, 2025c.
- Bellman, R. The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6):503–515, 1954.
- Bittanti, S., Laub, A. J., and Willems, J. C. The riccati equation. *Berlin and New York, Springer-Verlag, 1991, 347*, 1991.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- Dao, T. and Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- De Luca, A. B. and Fountoulakis, K. Simulation of graph algorithms with looped transformers. *arXiv preprint arXiv:2402.01107*, 2024.
- Dong, Q., Dong, L., Tang, Y., Ye, T., Sun, Y., Sui, Z., and Wei, F. Reinforcement pre-training. *arXiv preprint arXiv:2506.08007*, 2025.
- Dong, X., Fu, Y., Diao, S., Byeon, W., Chen, Z., Mahabaleshwarkar, A. S., Liu, S.-Y., Van Keirsbilck, M., Chen, M.-H., Suhara, Y., et al. Hymba: A hybrid-head architecture for small language models. *arXiv preprint arXiv:2411.13676*, 2024.
- Du, J., Sun, W., Lan, D., Hu, J., and Cheng, Y. Mom: Linear sequence modeling with mixture-of-memories. *arXiv preprint arXiv:2502.13685*, 2025.
- Du, S., Lee, J., Li, H., Wang, L., and Zhai, X. Gradient descent finds global minima of deep neural networks. In *International conference on machine learning*, pp. 1675–1685. PMLR, 2019.
- Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., and Goldstein, T. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.
- Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. In *International Conference on Machine Learning*, pp. 11398–11442. PMLR, 2023.
- Grazzi, R., Siems, J. N., Franke, J. K., Zela, A., Hutter, F., and Pontil, M. Unlocking state-tracking in linear rnns through negative eigenvalues. *International Conference on Learning Representations*, 2024.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

- Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33: 1474–1487, 2020.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021a.
- Gu, A., Johnson, I., Goel, K., Saab, K., Dao, T., Rudra, A., and Ré, C. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems*, 34: 572–585, 2021b.
- Gu, A., Goel, K., Gupta, A., and Ré, C. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 35: 35971–35983, 2022.
- Guha, E., Marten, R., Keh, S., Raoof, N., Smyrnis, G., Bansal, H., Nezhurina, M., Mercat, J., Vu, T., Sprague, Z., et al. Openthoughts: Data recipes for reasoning models. URL <https://arxiv.org/abs/2506.04178>, 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Gupta, A., Gu, A., and Berant, J. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*, 2022.
- Ha, D. and Schmidhuber, J. World models. *arXiv preprint arXiv:1803.10122*, 2(3), 2018.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.
- Hansen, N., Su, H., and Wang, X. Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., and Hu, Z. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Hatamizadeh, A., Akter, S. N., Prabhumoye, S., Kautz, J., Patwary, M., Shoenybi, M., Catanzaro, B., and Choi, Y. Rlp: Reinforcement as a pretraining objective. *arXiv preprint arXiv:2510.01265*, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Hopfield, J. J. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.
- Hopfield, J. J. Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the national academy of sciences*, 81(10): 3088–3092, 1984.
- Hopfield, J. J. and Tank, D. W. “neural” computation of decisions in optimization problems. *Biological cybernetics*, 52(3):141–152, 1985.
- Houlsby, N., Giurghi, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pp. 2790–2799. PMLR, 2019.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Kahneman, D. *Thinking, fast and slow*. macmillan, 2011.
- Kalman, R. E. et al. Contributions to the theory of optimal control. *Bol. soc. mat. mexicana*, 5(2):102–119, 1960.
- Kobayashi, S., Schimpf, Y., Schlegel, M., Steger, A., Wolczyk, M., von Oswald, J., Scherre, N., Maile, K., Lajoie, G., Richards, B. A., et al. Emergent temporal abstractions in autoregressive models enable hierarchical reinforcement learning. *arXiv preprint arXiv:2512.20605*, 2025.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Laub, A. A schur method for solving algebraic riccati equations. *IEEE Transactions on automatic control*, 24(6):913–921, 2003.

- LI, J., Beeching, E., Tunstall, L., Lipkin, B., Soletskyi, R., Huang, S. C., Rasul, K., Yu, L., Jiang, A., Shen, Z., Qin, Z., Dong, B., Zhou, L., Fleureau, Y., Lample, G., and Polu, S. Numina-math. [<https://huggingface.co/AI-MO/NuminaMath-CoT>] (https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.
- Liu, B., Wang, R., Wu, L., Feng, Y., Stone, P., and Liu, Q. Longhorn: State space models are amortized online learners. *arXiv preprint arXiv:2407.14207*, 2024.
- Long, J. Large language model guided tree-of-thought. *arXiv preprint arXiv:2305.08291*, 2023.
- McDuff, D. and Salamon, D. *Introduction to symplectic topology*. Oxford University Press, 2017.
- Nadaraya, E. A. On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142, 1964.
- Nhu, A. N., Son, S., and Lin, M. Time-aware world model for adaptive prediction and control. In *Forty-second International Conference on Machine Learning*, 2025.
- Palm, R., Paquet, U., and Winther, O. Recurrent relational networks. *Advances in neural information processing systems*, 31, 2018.
- Peng, B., Zhang, R., Goldstein, D., Alcaide, E., Du, X., Hou, H., Lin, J., Liu, J., Lu, J., Merrill, W., et al. Rwkv-7” goose” with expressive dynamic state evolution. *arXiv preprint arXiv:2503.14456*, 2025a.
- Peng, L., Chattopadhyay, A., Zancato, L., Nunez, E., Xia, W., and Soatto, S. Gated kalmanet: A fading memory layer through test-time ridge regression. *arXiv preprint arXiv:2511.21016*, 2025b.
- Ramsauer, H., Schöfl, B., Lehner, J., Seidl, P., Widrich, M., Adler, T., Gruber, L., Holzleitner, M., Pavlović, M., Sandve, G. K., et al. Hopfield networks is all you need. *arXiv preprint arXiv:2008.02217*, 2020.
- Ren, L., Liu, Y., Lu, Y., Shen, Y., Liang, C., and Chen, W. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.
- Saunshi, N., Dikkala, N., Li, Z., Kumar, S., and Reddi, S. J. Reasoning with latent thoughts: On the power of looped transformers. *arXiv preprint arXiv:2502.17416*, 2025.
- Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In *International conference on machine learning*, pp. 9355–9366. PMLR, 2021.
- Schmidhuber, J. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shojaee, P., Mirzadeh, I., Alizadeh, K., Horton, M., Bengio, S., and Farajtabar, M. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. *arxiv* 2025. *arXiv preprint arXiv:2506.06941*, 10, 2025.
- Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., Radford, A., Amodei, D., and Christiano, P. F. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33: 3008–3021, 2020.
- Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- Sun, Y., Li, X., Dalal, K., Xu, J., Vikram, A., Zhang, G., Dubois, Y., Chen, X., Wang, X., Koyejo, S., et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- Sutskever, I., Vinyals, O., and Le, Q. V. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.
- Tandon, A., Dalal, K., Li, X., Kocaja, D., Rød, M., Buchanan, S., Wang, X., Leskovec, J., Koyejo, S., Hashimoto, T., et al. End-to-end test-time training for long context. *arXiv preprint arXiv:2512.23675*, 2025.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is All You Need. In *Proceedings of NeurIPS*, 2017.
- Von Oswald, J., Schlegel, M., Meulemans, A., Kobayashi, S., Niklasson, E., Zucchet, N., Scherrer, N., Miller, N., Sandler, M., Vladymyrov, M., et al. Uncovering mesa-optimization algorithms in transformers. *arXiv preprint arXiv:2309.05858*, 2023.

- von Oswald, J., Scherrer, N., Kobayashi, S., Versari, L., Yang, S., Schlegel, M., Maile, K., Schimpf, Y., Sieberling, O., Meulemans, A., et al. Mesanet: Sequence modeling by locally optimal test-time training. *arXiv preprint arXiv:2506.05233*, 2025.
- Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S., and Yadkori, Y. A. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025a.
- Wang, K. A., Shi, J., and Fox, E. B. Test-time regression: a unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025b.
- Wang, P.-W., Donti, P., Wilder, B., and Kolter, Z. Satnet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver. In *International Conference on Machine Learning*, pp. 6545–6554. PMLR, 2019.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Widrich, M., Schäfl, B., Pavlović, M., Ramsauer, H., Gruber, L., Holzleitner, M., Brandstetter, J., Sandve, G. K., Greiff, V., Hochreiter, S., et al. Modern hopfield networks and attention for immune repertoire classification. *Advances in neural information processing systems*, 33:18832–18845, 2020.
- Yang, L., Lee, K., Nowak, R., and Papailiopoulos, D. Looped transformers are better at learning learning algorithms. *arXiv preprint arXiv:2311.12424*, 2023a.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023b.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024a.
- Yang, S., Wang, B., Zhang, Y., Shen, Y., and Kim, Y. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024b.
- Yang, Z., Ishay, A., and Lee, J. Learning to solve constraint satisfaction problems with recurrent transformer. *arXiv preprint arXiv:2307.04895*, 2023c.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? URL <https://arxiv.org/abs/2504.13837>, 2025.
- Yuksekgonul, M., Kocejka, D., Li, X., Bianchi, F., McCaleb, J., Wang, X., Kautz, J., Choi, Y., Zou, J., Guestrin, C., et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.
- Zancato, L., Seshadri, A., Dukler, Y., Golatkar, A. S., Shen, Y., Bowman, B., Trager, M., Achille, A., and Soatto, S. B’mojo: Hybrid state space realizations of foundation models with eidetic and fading memory. *Advances in Neural Information Processing Systems*, 37:130433–130462, 2024.
- Zeng, B., Song, S., Huang, S., Wang, Y., Li, H., He, Z., Wang, X., Li, Z., and Lin, Z. Pretraining language models to ponder in continuous space. *arXiv preprint arXiv:2505.20674*, 2025.
- Zhang, T., Bi, S., Hong, Y., Zhang, K., Luan, F., Yang, S., Sunkavalli, K., Freeman, W. T., and Tan, H. Test-time training done right. *arXiv preprint arXiv:2505.23884*, 2025.
- Zhu, H., Hao, S., Hu, Z., Jiao, J., Russell, S., and Tian, Y. Emergence of superposition: Unveiling the training dynamics of chain of continuous thought. *arXiv preprint arXiv:2509.23365*, 2025a.
- Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025b.
- Zuo, Y., Zhang, K., Sheng, L., Qu, S., Cui, G., Zhu, X., Li, H., Zhang, Y., Long, X., Hua, E., et al. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.

A. Theory

Definition A.1 (Linear-Quadratic Regulator (Kalman et al., 1960)). Given LQR system with parameters $\{(A_t, B_t, Q_t, R_t, r_t)\}_{t=1}^T$ and initial state $h_0 = h_{init}$, the optimization problem can be formally written as:

$$\min_{\substack{h_0, h_1, \dots, h_T \\ u_1, \dots, u_T}} \left[\frac{1}{2} (h_t^\top Q_t h_t + u_t^\top R_t u_t) + r_t^\top u_t \right] \quad (15)$$

$$\text{s.t. } h_{t+1} = A_{t+1} h_t + B_{t+1} u_{t+1}, \quad h_0 = h_{init} \quad (16)$$

Note that we distinguish between the notation for h_0 as an optimization variable and its assigned value h_{init} . In the main text, we use h_0 in place of h_{init} without separate notations when no confusion arises.

Below, we introduce two approaches to solve Eq. (15), along the way, proving Proposition 3.1 and Theorem 3.3. Afterwards, we conclude this section by proving Theorem 3.2.

A.1. Riccati Iteration

One of the most famous algorithms to solve Eq. (15) is through Riccati iteration algorithm, grounded by the following proposition:

Proposition A.2. *The optimal u_t minimizing Eq. (8) depends affinely on h_{t-1} as $u_t = K_t h_{t-1} + k_t$, where K_t and k_t are given by:*

$$K_t = -(R_t + B_t^\top P_t B_t)^{-1} B_t^\top P_t A_t, \quad k_t = -(R_t + B_t^\top P_t B_t)^{-1} (B_t^\top p_t + r_t), \quad (17)$$

where:

$$P_t = Q_t + A_{t+1}^\top P_{t+1} A_{t+1} - (B_{t+1}^\top P_{t+1} A_{t+1})^\top (R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1})^{-1} (B_{t+1}^\top P_{t+1} A_{t+1}), \quad (18)$$

$$p_t = A_{t+1}^\top p_{t+1} - A_{t+1}^\top P_{t+1} B_{t+1} (R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1})^{-1} (B_{t+1}^\top p_{t+1} + r_{t+1}), \quad (19)$$

for $t = 1, \dots, T$ with boundary condition $P_T = Q_T$ and $p_T = 0$.

Proof. First, define the value function $V_t(h)$ as the minimal cost-to-go given $h_t = h$ for any $t \geq 0$:

$$V_t(h) = \min_{u_{t+1}, \dots, u_T} \frac{1}{2} \left[h_t^\top Q_t h_t + \sum_{s=1}^{T-t} (h_{t+s}^\top Q_{t+s} h_{t+s} + u_{t+s}^\top R_{t+s} u_{t+s} + 2r_{t+s}^\top u_{t+s}) \right]. \quad (20)$$

Unrolling the optimization problem in Eq. (20) by one step yields the Bellman equation:

$$V_t(h) = \min_u \left[\frac{1}{2} h^\top Q_t h + \frac{1}{2} u^\top R_{t+1} u + r_{t+1}^\top u + V_{t+1}(A_{t+1} h + B_{t+1} u) \right].$$

Next, we make the inductive hypothesis that $V_{t+1}(h) = \frac{1}{2} h^\top P_{t+1} h + p_{t+1}^\top h + c_{t+1}$ for some symmetric matrix $P_{t+1} \in \mathbb{R}^{d \times d}$, vector $p_{t+1} \in \mathbb{R}^d$, and scalar $c_{t+1} \in \mathbb{R}$. This hypothesis holds for $t = T$: $V_T(h) = h^\top Q_T h$. Substituting the hypothesis into the Bellman equation gives

$$\begin{aligned} V_t(h) &= \min_u \left[\frac{1}{2} h^\top Q_t h + \frac{1}{2} u^\top R_{t+1} u + r_{t+1}^\top h + \frac{1}{2} (A_{t+1} h + B_{t+1} u)^\top P_{t+1} (A_{t+1} h + B_{t+1} u) \right. \\ &\quad \left. + p_{t+1}^\top (A_{t+1} h + B_{t+1} u) + c_{t+1} \right] \\ &= \min_u \frac{1}{2} \left[h^\top (Q_t + A_{t+1}^\top P_{t+1} A_{t+1}) h + u^\top (R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1}) u \right. \\ &\quad \left. + 2h^\top A_{t+1}^\top P_{t+1} B_{t+1} u + 2(B_{t+1}^\top p_{t+1} + r_{t+1})^\top u + 2p_{t+1}^\top A_{t+1} h + 2c_{t+1} \right]. \end{aligned}$$

The minimizer admits the closed form

$$u_{t+1} = -(R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1})^{-1} (B_{t+1}^\top P_{t+1} A_{t+1} h + B_{t+1}^\top p_{t+1} + r_{t+1}), \quad (21)$$

and the minimum value is

$$\begin{aligned} V_t(h) = & \frac{1}{2} h^\top \left[Q_t + A_{t+1}^\top P_{t+1} A_{t+1} - (B_{t+1}^\top P_{t+1} A_{t+1})^\top (R_{t+1} + B_t^\top P_{t+1} B_t)^{-1} (B_{t+1}^\top P_{t+1} A_{t+1}) \right] h \\ & + \left[A_{t+1}^\top p_{t+1} - A_{t+1}^\top P_{t+1} B_{t+1} (R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1})^{-1} (B_{t+1}^\top p_{t+1} + r_{t+1}) \right]^\top h \\ & + c_{t+1} - \frac{1}{2} (B_{t+1}^\top p_{t+1} + r_{t+1})^\top (R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1})^{-1} (B_{t+1}^\top p_{t+1} + r_{t+1}). \end{aligned}$$

This completes the induction by identifying P_t, p_t, c_t for V_t as desired. $\square$

Remark A.3. Proposition 3.1 is a special case of Theorem A.2 when considering $t = 1$ and no affine cost on actions. Solving an LQR with a linear cost upon actions is needed for backward (Sec. 3.2).

Theorem A.2 implies Algorithm 1 to compute all $\{(h_t, u_t)\}_{t=1}^T$.

Algorithm 1 Finite-horizon LQR via Riccati recursion

Require: $\{(A_t, B_t, Q_t, R_t, r_t)\}_{t=1}^T$, initial state h_{init}

Ensure: actions $\{u_t\}_{t=1}^T$, states $\{h_t\}_{t=1}^T$

- 1: Initialize terminal parameters $P_T \leftarrow Q_T, p_T \leftarrow 0$
 - 2: **for** $t = T - 1, \dots, 1$ **do** $\triangleright$ reverse Riccati recursion
 - 3: $P_t \leftarrow Q_t + A_{t+1}^\top P_{t+1} A_{t+1} - (B_{t+1}^\top P_{t+1} A_{t+1})^\top (R_{t+1} + B_t^\top P_{t+1} B_t)^{-1} (B_{t+1}^\top P_{t+1} A_{t+1})$
 - 4: $p_t \leftarrow A_{t+1}^\top p_{t+1} - A_{t+1}^\top P_{t+1} B_{t+1} (R_{t+1} + B_{t+1}^\top P_{t+1} B_{t+1})^{-1} (B_{t+1}^\top p_{t+1} + r_{t+1})$
 - 5: **end for**
 - 6: Assign initial state $h_0 \leftarrow h_{init}$.
 - 7: **for** $t = 1, 2, \dots, T$ **do** $\triangleright$ forward rollout
 - 8: $u_t \leftarrow -(R_t + B_t^\top P_t B_t)^{-1} (B_t^\top P_t A_t h_{t-1} + B_t^\top p_t + r_t)$
 - 9: $h_t \leftarrow A_t h_{t-1} + B_t u_t$
 - 10: **end for**
 - 11: **return** $\{u_t\}_{t=1}^T, \{h_t\}_{t=1}^T$.
-

A.2. KKT Reformulation

We introduce Lagrangian multiplier $\{\lambda_t\}_{0 \leq t \leq T}$, and the objective Eq. (15) can be transformed to:

$$L = \frac{1}{2} \sum_{t=1}^T (h_t^\top Q_t h_t + u_t^\top R_t u_t) + \sum_{t=1}^T r_t^\top u_t + \sum_{t=0}^{T-1} \lambda_{t+1}^\top (A_{t+1} h_t + B_{t+1} u_{t+1} - h_{t+1}) + \lambda_0^\top (h_{init} - h_0), \quad (22)$$

wherer λ_t is also called co-state. The dual problem to Eq. (15) can then be written as (Boyd & Vandenberghe, 2004):

$$\max_{\lambda_0, \dots, \lambda_T} \inf_{\{(h_t, u_t)\}_{t=1}^T} L(\{(h_t, \lambda_t, u_t)\}_{t=1}^T). \quad (23)$$

By KKT conditions (Boyd & Vandenberghe, 2004), the optimal solution $\{(h_t, \lambda_t, u_t)\}_{t=1}^T$ to Eq. (23) must satisfy:

$$\frac{\partial L}{\partial h_t} = Q_t h_t + A_{t+1}^\top \lambda_{t+1} - \lambda_t = 0, \quad 1 \leq t \leq T - 1, \quad (24)$$

$$\frac{\partial L}{\partial h_t} = A_{t+1}^\top \lambda_{t+1} - \lambda_t = 0, \quad t = 0, \quad (25)$$

$$\frac{\partial L}{\partial h_t} = Q_t h_t - \lambda_t = 0, \quad t = T \quad (26)$$

$$\frac{\partial L}{\partial u_t} = R_t u_t + B_t^\top \lambda_t + r_t = 0, \quad 1 \leq t \leq T, \quad (27)$$

$$\frac{\partial L}{\partial \lambda_t} = A_t h_{t-1} + B_t u_t - h_t = 0, \quad 1 \leq t \leq T, \quad (28)$$

$$\frac{\partial L}{\partial \lambda_t} = h_{init} - h_t = 0, \quad t = 0. \quad (29)$$

Combining Riccati iteration, Eq. (24), (25) and (26) imply a way to compute all $\{\lambda_t\}_{t=0}^T$ from $\{(h_t, u_t)\}_{t=1}^T$. See Algorithm 2 for more details.

Algorithm 2 Extension to Riccati recursion

Require: $\{(A_t, B_t, Q_t, R_t, r_t)\}_{t=1}^T$, actions $\{u_t\}_{t=1}^T$, states $\{h_t\}_{t=0}^T$,

Ensure: co-states $\{\lambda_t\}_{t=0}^T$ ▷ Eq. (26)

1: Initialize terminal co-state $\lambda_T \leftarrow Q_T h_T$

2: **for** $t = T - 1, \dots, 1$ **do**

3: $\lambda_t \leftarrow Q_t h_t + A_{t+1}^\top \lambda_{t+1}$ ▷ Eq. (24)

4: **end for**

5: $\lambda_0 \leftarrow A_1^\top \lambda_1$ ▷ Eq. (25)

6: **return** $\{\lambda_t\}_{t=0}^T$.

A.3. Symplectic Iteration

We prove the symplectic iteration theorem in two parts. First, we establish the joint relation between states and co-states, which is useful for forward computation of all states and co-states starting from h_0 and λ_0 .

Theorem A.4 (Forward symplectic iteration). *Given LQR system in Eq. (15) with parameters $\{(A_t, B_t, Q_t, R_t, r_t)\}_{t=1}^T$ and initial state h_{init} . The following recursive relation between state h_t and co-state λ_t is always true for $1 \leq t \leq T$:*

$$\begin{bmatrix} h_t \\ \lambda_t \end{bmatrix} = S_t \begin{bmatrix} h_{t-1} \\ \lambda_{t-1} \end{bmatrix} + b_t, \quad S_t = \begin{bmatrix} A_t + G_t(A_t^\top)^{-1}Q_{t-1} & -G_t(A_t^\top)^{-1} \\ -(A_t^\top)^{-1}Q_{t-1} & (A_t^\top)^{-1} \end{bmatrix}, \quad b_t = \begin{bmatrix} -B_t R_t^{-1} r_t \\ 0 \end{bmatrix}, \quad (30)$$

where we define $Q_0 = 0$ and $G_t = B_t R_t^{-1} B_t^\top$.

Proof. Starting with KKT condition Eq. (27), we have $u_t = -R_t^{-1}(B_t^\top \lambda_t + r_t)$. After re-organizing Eq. (28) and (24):

$$h_t = A_t h_{t-1} - B_t R_t^{-1} B_t^\top \lambda_t - B_t R_t^{-1} r_t, \quad 1 \leq t \leq T, \quad (31)$$

$$\lambda_t = Q_t h_t + A_{t+1}^\top \lambda_{t+1} \quad 1 \leq t \leq T-1, \quad (32)$$

with boundary condition:

$$\lambda_0 = A_1^\top \lambda_1, \quad \lambda_T = Q_T h_T, \quad h_0 = h_{init}. \quad (33)$$

By Eq. (32) above, we have:

$$\lambda_t = (A_t^\top)^{-1}(\lambda_{t-1} - Q_{t-1} h_{t-1}),$$

and henceforth

$$\begin{aligned} h_t &= A_t h_{t-1} - B_t R_t^{-1} B_t^\top (A_t^\top)^{-1}(\lambda_{t-1} - Q_{t-1} h_{t-1}) - B_t R_t^{-1} r_t \\ &= (A_t + G_t(A_t^\top)^{-1}Q_{t-1})h_{t-1} - G_t(A_t^\top)^{-1}\lambda_{t-1} - g_t, \end{aligned}$$

for $2 \leq t \leq T$, where we denote $G_t = B_t R_t^{-1} B_t^\top$, $g_t = B_t R_t^{-1} r_t$ to avoid notational clusters.

We define $Q_0 = 0$, and by Eq. (33), we can derive a similar recurrent form for the initial step:

$$\lambda_1 = (A_1^\top)^{-1}\lambda_0 = (A_1^\top)^{-1}(\lambda_0 - Q_0 h_0),$$

and

$$\begin{aligned} h_1 &= A_1 h_0 + B_1 u_1 = A_1 h_0 - G_1 \lambda_1 - g_1 = A_1 h_0 - G_1 (A_1^\top)^{-1} \lambda_0 - g_1 \\ &= (A_1 + G_1 (A_1^\top)^{-1} Q_0) h_0 + G_1 (A_1^\top)^{-1} \lambda_0 - g_1. \end{aligned}$$

After rewriting above equations in matrix form, we obtain the joint update rule for $[h_t, \lambda_t]$ as desired. □

Second, we show the reverse symplectic iteration theorem that is useful to compute the initial co-state λ_0 together with the optimal first-step action. The result below directly proves Theorem 3.3 in the main text.

Theorem A.5 (Reverse symplectic iteration). *Given LQR system in Eq. (15) with parameters $\{(A_t, B_t, Q_t, R_t, r_t)\}_{t=1}^T$ and initial state h_{init} . The optimal first-step control:*

$$u_1 = -R_1^{-1}(B_1^\top(A_1^\top)^{-1}Y_1^{-1}(Y_2 h_{init} + y_3) + r_1), \quad \lambda_0 = Y_1^{-1}(Y_2 h_{init} + y_3)$$

where ³

$$[Y_1 \quad Y_2] = [I \quad Q_T] \prod_{t=1}^T \Sigma_t, \quad y_3 = [I \quad Q_T] \left(\sum_{t=1}^T \left(\prod_{r=t+1}^T \Sigma_r \right) \beta_t \right),$$

$$\Sigma_t = \begin{bmatrix} (A_t^\top)^{-1} & (A_t^\top)^{-1}Q_{t-1} \\ G_t(A_t^\top)^{-1} & A_t + G_t(A_t^\top)^{-1}Q_{t-1} \end{bmatrix}, \quad \beta_t = \begin{bmatrix} 0 \\ -B_t R_t^{-1} r_t \end{bmatrix}, \quad G_t = B_t R_t^{-1} B_t^\top,$$

Proof. By Theorem A.4, the relation between h_0 and λ_0 can be found by the following “shooting” system:

$$\begin{bmatrix} h_T \\ \lambda_T \end{bmatrix} = \left(\prod_{t=1}^T S_t \right) \begin{bmatrix} h_0 \\ \lambda_0 \end{bmatrix} + \sum_{t=1}^T \left(\prod_{r=t+1}^T S_r \right) b_t, \quad Q_T h_T = \lambda_T. \quad (34)$$

Now we define: $\Psi = \begin{bmatrix} \Psi_{11} & \Psi_{12} \\ \Psi_{21} & \Psi_{22} \end{bmatrix} = \prod_{t=1}^T S_t$, and $\psi = [\psi_1^\top, \psi_2^\top]^\top = \sum_{t=1}^T \left(\prod_{r=t+1}^T S_r \right) b_t$. The above system can be rewritten as:

$$\begin{aligned} 0 &= Q_T(\Psi_{11} h_0 + \Psi_{12} \lambda_0 + \psi_1) - (\Psi_{21} h_0 + \Psi_{22} \lambda_0 + \psi_2) \\ &= (Q_T \Psi_{11} - \Psi_{21}) h_0 + (Q_T \Psi_{12} - \Psi_{22}) \lambda_0 + (Q_T \psi_1 - \psi_2). \end{aligned}$$

This shows: $\lambda_0 = -(Q_T \Psi_{12} - \Psi_{22})^{-1}(Q_T \Psi_{11} - \Psi_{21}) h_0 + (Q_T \Psi_{12} - \Psi_{22})^{-1}(Q_T \psi_1 - \psi_2)$. In fact, there is no need to compute every block of Ψ . Instead, we can show that: $\lambda_0 = Y_1^{-1}(Y_2 h_0 + y_3)$ where:

$$\begin{aligned} [Y_1 \quad Y_2] &= [I \quad Q_T] \begin{bmatrix} \Psi_{22} & -\Psi_{21} \\ -\Psi_{12} & \Psi_{11} \end{bmatrix} = [I \quad Q_T] (\Psi^{-1})^\top \\ &= [I \quad Q_T] \left(\left(\prod_{t=1}^T S_t \right)^{-1} \right)^\top = [I \quad Q_T] \left(\prod_{t=1}^T S_{T-t+1}^{-1} \right)^\top \\ &= [I \quad Q_T] \prod_{t=1}^T (S_t^{-1})^\top, \end{aligned}$$

The second equality are due to Lemma A.6: S_t is symplectic for every $1 \leq t \leq T$, so is their product. Similarly, we can

³Note that, different from conventional notation, we denote $\prod_{t=1}^T \Sigma_t = \Sigma_T \Sigma_{T-1} \cdots \Sigma_2 \Sigma_1$ to simplify subscripts, where matrices are cumulatively multiplied to the left (instead of right).

simplify the calculation of $\mathbf{y}_3$ as below:

$$\begin{aligned}
 \mathbf{y}_3 &= \begin{bmatrix} \mathbf{I} & \mathbf{Q}_T \end{bmatrix} \begin{bmatrix} -\psi_2 \\ \psi_1 \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{Q}_T \end{bmatrix} \mathbf{J}^\top \psi \\
 &= \begin{bmatrix} \mathbf{I} & \mathbf{Q}_T \end{bmatrix} \mathbf{J}^\top \left(\sum_{t=1}^T \left(\prod_{r=t+1}^T \mathbf{S}_r \right) \mathbf{b}_t \right) \\
 &= \begin{bmatrix} \mathbf{I} & \mathbf{Q}_T \end{bmatrix} \left(\sum_{t=1}^T \left[\left(\prod_{r=t+1}^T \mathbf{S}_r \right)^{-1} \right]^\top \mathbf{J}^\top \mathbf{b}_t \right) \\
 &= \begin{bmatrix} \mathbf{I} & \mathbf{Q}_T \end{bmatrix} \left(\sum_{t=1}^T \left(\prod_{r=t+1}^T (\mathbf{S}_r^{-1})^\top \right) \mathbf{J}^\top \mathbf{b}_t \right),
 \end{aligned}$$

where $\mathbf{J} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{bmatrix}$ is the standard symplectic matrix. The fourth equality is due to symplecticity.

We let $\beta_t = \mathbf{J}^\top \mathbf{b}_t = [\mathbf{0}^\top, -\mathbf{g}_t^\top]^\top$. Using symplecticity again, we can have the closed form of $(\mathbf{S}_t^{-1})^\top$:

$$\Sigma_t := (\mathbf{S}_t^{-1})^\top = \begin{bmatrix} (\mathbf{A}_t^\top)^{-1} & (\mathbf{A}_t^\top)^{-1} \mathbf{Q}_{t-1} \\ \mathbf{G}_t (\mathbf{A}_t^\top)^{-1} & \mathbf{A}_t + \mathbf{G}_t (\mathbf{A}_t^\top)^{-1} \mathbf{Q}_{t-1} \end{bmatrix}.$$

After solving λ_0 with $\mathbf{h}_0 = \mathbf{h}_{init}$, we can obtain $\mathbf{u}_1$ by the KKT condition Eq. (27):

$$\mathbf{u}_1 = -\mathbf{R}_1^{-1} (\mathbf{B}_1^\top \lambda_1 + \mathbf{r}_1) = -\mathbf{R}_1^{-1} (\mathbf{B}_1^\top (\mathbf{A}_1^\top)^{-1} \lambda_0 + \mathbf{r}_1).$$

□

Lemma A.6. $\mathbf{S}_t$ is a symplectic matrix: $-\mathbf{J} \mathbf{S}_t^\top \mathbf{J} = \mathbf{S}_t^{-1}$, where $\mathbf{J} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{bmatrix}$.

Proof. Consider a matrix of the following form (with subscripts removed):

$$\mathbf{S} = \begin{bmatrix} \mathbf{A} + \mathbf{G}(\mathbf{A}^\top)^{-1} \mathbf{Q} & -\mathbf{G}(\mathbf{A}^\top)^{-1} \\ -(\mathbf{A}^\top)^{-1} \mathbf{Q} & (\mathbf{A}^\top)^{-1} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_s & \mathbf{B}_s \\ \mathbf{C}_s & \mathbf{D}_s \end{bmatrix},$$

where $\mathbf{A}$ is invertible and $\mathbf{Q}^\top = \mathbf{Q}$, $\mathbf{G}^\top = \mathbf{G}$.

$\mathbf{S}$ being symplectic is equivalent to say $\mathbf{S}^\top \mathbf{J} \mathbf{S} = \mathbf{J}$. This is further equivalent to the blockwise conditions:

$$\mathbf{A}_s^\top \mathbf{C}_s = \mathbf{C}_s^\top \mathbf{A}_s, \quad \mathbf{B}_s^\top \mathbf{D}_s = \mathbf{D}_s^\top \mathbf{B}_s, \quad \mathbf{A}_s^\top \mathbf{D}_s - \mathbf{C}_s^\top \mathbf{B}_s = \mathbf{I}.$$

Then we verify that $\mathbf{A}_s^\top \mathbf{C}_s = \mathbf{C}_s^\top \mathbf{A}_s$:

$$\begin{aligned}
 \mathbf{A}_s^\top \mathbf{C}_s &= (\mathbf{A}^\top + \mathbf{Q} \mathbf{A}^{-1} \mathbf{G})(-\mathbf{A}^\top)^{-1} \mathbf{Q} = -\mathbf{A}^\top (\mathbf{A}^\top)^{-1} \mathbf{Q} - \mathbf{Q} \mathbf{A}^{-1} \mathbf{G} (\mathbf{A}^\top)^{-1} \mathbf{Q} = -\mathbf{Q} - \mathbf{Q} \mathbf{A}^{-1} \mathbf{G} (\mathbf{A}^\top)^{-1} \mathbf{Q} \\
 &= -\mathbf{Q} \mathbf{A}^{-1} \mathbf{A} - \mathbf{Q} \mathbf{A}^{-1} \mathbf{G} (\mathbf{A}^\top)^{-1} \mathbf{Q} = (-\mathbf{Q} \mathbf{A}^{-1})(\mathbf{A} + \mathbf{G} (\mathbf{A}^\top)^{-1} \mathbf{Q}) = \mathbf{C}_s^\top \mathbf{A}_s,
 \end{aligned}$$

using the fact that $\mathbf{Q}$ and $\mathbf{G}$ are symmetric. Similarly, $\mathbf{B}_s^\top \mathbf{D}_s = \mathbf{D}_s^\top \mathbf{B}_s$ due to associativity:

$$\mathbf{B}_s^\top \mathbf{D}_s = (-\mathbf{A}^{-1} \mathbf{G})(\mathbf{A}^\top)^{-1} = -\mathbf{A}^{-1} \mathbf{G} (\mathbf{A}^\top)^{-1} = \mathbf{D}_s^\top \mathbf{B}_s.$$

The last step is to verify $\mathbf{A}_s^\top \mathbf{D}_s - \mathbf{C}_s^\top \mathbf{B}_s = \mathbf{I}$:

$$\begin{aligned}
 \mathbf{A}_s^\top \mathbf{D}_s &= (\mathbf{A}^\top + \mathbf{Q} \mathbf{A}^{-1} \mathbf{G})(\mathbf{A}^\top)^{-1} = \mathbf{I} + \mathbf{Q} \mathbf{A}^{-1} \mathbf{G} (\mathbf{A}^\top)^{-1}, \\
 \mathbf{C}_s^\top \mathbf{B}_s &= (-\mathbf{Q} \mathbf{A}^{-1})(-\mathbf{G} (\mathbf{A}^\top)^{-1}) = \mathbf{Q} \mathbf{A}^{-1} \mathbf{G} (\mathbf{A}^\top)^{-1}.
 \end{aligned}$$

Therefore, $\mathbf{A}_s^\top \mathbf{D}_s - \mathbf{C}_s^\top \mathbf{B}_s = \mathbf{I}$.

□

Algorithm 3 Finite-horizon LQR via symplectic iteration

Require: $\{(A_t, B_t, Q_t, R_t, r_t)\}_{t=1}^T$, initial state h_{init}
Ensure: actions $\{u_t\}_{t=1}^T$, states $\{h_t\}_{t=1}^T$, co-states $\{\lambda_t\}_{t=0}^T$

- 1: Initialize $[Y_1 \ Y_2] \leftarrow [I \ Q_T]$, $y_3 \leftarrow 0$.
- 2: **for** $t = T, \dots, 1$ **do** ▷ reverse symplectic iteration
- 3: $\Sigma_t \leftarrow \begin{bmatrix} (A_t^\top)^{-1} & (A_t^\top)^{-1}Q_{t-1} \\ G_t(A_t^\top)^{-1} & A_t + G_t(A_t^\top)^{-1}Q_{t-1} \end{bmatrix}$
- 4: $[Y_1 \ Y_2] \leftarrow [Y_1 \ Y_2] \Sigma_t$
- 5: $y_3 \leftarrow \Sigma_t y_3 + \begin{bmatrix} 0 \\ -B_t R_t^{-1} r_t \end{bmatrix}$
- 6: **end for**
- 7: $y_3 \leftarrow [I \ Q_T] y_3$
- 8: Assign initial state $h_0 \leftarrow h_{init}$
- 9: Compute initial co-state $\lambda_0 \leftarrow Y_1^{-1}(Y_2 h_{init} + y_3)$
- 10: **for** $t = 1, 2, \dots, T$ **do** ▷ forward symplectic iteration
- 11: $\begin{bmatrix} h_t \\ \lambda_t \end{bmatrix} \leftarrow \begin{bmatrix} A_t + G_t(A_t^\top)^{-1}Q_{t-1} & -G_t(A_t^\top)^{-1} \\ -(A_t^\top)^{-1}Q_{t-1} & (A_t^\top)^{-1} \end{bmatrix} \begin{bmatrix} h_{t-1} \\ \lambda_{t-1} \end{bmatrix} + \begin{bmatrix} -B_t R_t^{-1} r_t \\ 0 \end{bmatrix}$
- 12: $u_t \leftarrow -R_t^{-1}(B_t^\top \lambda_t + r_t)$
- 13: **end for**
- 14: **return** $\{u_t\}_{t=1}^T, \{h_t\}_{t=1}^T, \{\lambda_t\}_{t=0}^T$.

Remark A.7 (Complexity analysis). Solving LQR with either Riccati or symplectic iteration has the same theoretical computational complexity $O(Td^3)$ and memory complexity $O(d^2)$.

Theorems A.4 and A.5 lead to a new algorithm for solving LQRs, as illustrated in Algorithm 3.

Remark A.8. Solving LQR with Algorithm 3 only requires one reverse and one forward iteration to solve all $\{(h_t, \lambda_t, u_t)\}_{t=1}^T$ and λ_0 while Algorithm 1 and 2 need one forward iteration and two reverse iterations.

A.4. Differentiation through KKT Conditions

Below, we formally prove Theorem 3.2. The proof technique is based on Amos & Kolter (2017).

Theorem A.9. Consider a TTC layer with parameters $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$ and initial state h_{init} , suppose we have output $u_{out} = \text{TTC}(h_{init}, \{(A_t, B_t, Q_t, R_t)\}_{t=1}^T)$, the loss is computed as $\ell(u_{out})$, and we have obtained $\nabla_{u_{out}} \ell$. Then the gradients w.r.t. $\{(A_t, B_t, Q_t, R_t)\}$ are:

$$\begin{aligned} \nabla_{A_t} \ell &= \lambda_t \tilde{h}_{t-1}^\top + \tilde{\lambda}_t h_{t-1}^\top, & \nabla_{B_t} \ell &= \lambda_t \tilde{u}_t^\top + \tilde{\lambda}_t u_t^\top, \\ \nabla_{Q_t} \ell &= \frac{1}{2}(h_t \tilde{h}_t^\top + \tilde{h}_t h_t^\top), & \nabla_{R_t} \ell &= \frac{1}{2}(u_t \tilde{u}_t^\top + \tilde{u}_t u_t^\top), \end{aligned}$$

and gradient w.r.t. h_{init} can be computed by $\nabla_{h_{init}} \ell = \tilde{\lambda}_0$, where $\{(h_0, \lambda_0)\} \cup \{(h_t, \lambda_t, u_t)\}_{t=1}^T$ are the solution to the KKT system associated with Eq. (8) and $\{(\tilde{h}_0, \tilde{\lambda}_0)\} \cup \{(\tilde{h}_t, \tilde{\lambda}_t, \tilde{u}_t)\}_{t=1}^T$ are the solution to the KKT system corresponding to the following LQR system with $\tilde{h}_0 = 0$:

$$\min \frac{1}{2} \sum_{t=1}^T (\tilde{h}_t^\top Q_t \tilde{h}_t + \tilde{u}_t^\top R_t \tilde{u}_t) + \nabla_{u_{out}} \ell^\top \tilde{u}_1 \quad (35)$$

$$\text{s.t. } \tilde{h}_t = A_t \tilde{h}_{t-1} + B_t \tilde{u}_t, \quad 1 \leq t \leq T. \quad (36)$$

Proof. To see this more easily, we consider a matrix form of the KKT system in Eq. (15):

$$L = \frac{1}{2} x^\top C x + \xi^\top (F x - b)$$

where we concatenate all states and actions in $\mathbf{x} = [\mathbf{h}_0^\top, \dots, \mathbf{h}_T^\top, \mathbf{u}_1, \dots, \mathbf{u}_T]^\top \in \mathbb{R}^{(2T+1)d}$, all co-states in $\boldsymbol{\xi} = [\boldsymbol{\lambda}_0^\top, \dots, \boldsymbol{\lambda}_T^\top]^\top \in \mathbb{R}^{(T+1)d}$, and define cost matrix $\mathbf{C} \in \mathbb{R}^{(2T+1)d \times (2T+1)d}$ and linear constraints $\mathbf{F} \in \mathbb{R}^{(T+1)d \times (2T+1)d}$ as below⁴:

$$\mathbf{C} = \begin{bmatrix} \mathbf{0} & & & & \\ & \frac{\mathbf{Q}_1 + \mathbf{Q}_1^\top}{2} & & & \\ & & \ddots & & \\ & & & \frac{\mathbf{R}_1 + \mathbf{R}_1^\top}{2} & \\ & & & & \ddots \end{bmatrix}, \quad \mathbf{F} = \left[\begin{array}{ccccc|cccc} -\mathbf{I} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{A}_1 & -\mathbf{I} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{B}_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_2 & -\mathbf{I} & \cdots & \mathbf{0} & \mathbf{0} & \mathbf{B}_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{A}_T & -\mathbf{I} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{B}_T \end{array} \right], \quad \mathbf{b} = \begin{bmatrix} -\mathbf{h}_{init} \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}.$$

The Eqs. (24)-(29) can be expressed as:

$$\nabla_{\mathbf{x}} L = \mathbf{C}\mathbf{x} + \mathbf{F}^\top \boldsymbol{\xi} = \mathbf{0}, \quad \nabla_{\boldsymbol{\xi}} L = \mathbf{F}\mathbf{x} - \mathbf{b} = \mathbf{0}.$$

Consider a scalar parameter θ from any of $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T$ or $\mathbf{h}_{init}$, we take derivatives w.r.t. θ from both sides of the above equations:

$$\frac{\partial}{\partial \theta} \mathbf{C}\mathbf{x} + \mathbf{C} \frac{\partial}{\partial \theta} \mathbf{x} + \frac{\partial}{\partial \theta} \mathbf{F}^\top \boldsymbol{\xi} + \mathbf{F}^\top \frac{\partial}{\partial \theta} \boldsymbol{\xi} = \mathbf{0}, \quad \frac{\partial}{\partial \theta} \mathbf{F}\mathbf{x} + \mathbf{F} \frac{\partial}{\partial \theta} \mathbf{x} - \frac{\partial}{\partial \theta} \mathbf{b} = \mathbf{0},$$

which can be rewritten in the following matrix form:

$$\begin{bmatrix} \mathbf{C} & \mathbf{F}^\top \\ \mathbf{F} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{x} \\ \frac{\partial}{\partial \theta} \boldsymbol{\xi} \end{bmatrix} = \begin{bmatrix} -\frac{\partial}{\partial \theta} \mathbf{C}\mathbf{x} - \frac{\partial}{\partial \theta} \mathbf{F}^\top \boldsymbol{\xi} \\ -\frac{\partial}{\partial \theta} \mathbf{F}\mathbf{x} + \frac{\partial}{\partial \theta} \mathbf{b} \end{bmatrix}. \quad (37)$$

This implies that we can solve the following equation to obtain $\left[\frac{\partial}{\partial \theta} \mathbf{x}^\top, \frac{\partial}{\partial \theta} \boldsymbol{\xi}^\top \right]^\top$:

$$\begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{x} \\ \frac{\partial}{\partial \theta} \boldsymbol{\xi} \end{bmatrix} = - \begin{bmatrix} \mathbf{C} & \mathbf{F}^\top \\ \mathbf{F} & \mathbf{0} \end{bmatrix}^{-1} \begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{C}\mathbf{x} + \frac{\partial}{\partial \theta} \mathbf{F}^\top \boldsymbol{\xi} \\ \frac{\partial}{\partial \theta} \mathbf{F}\mathbf{x} - \frac{\partial}{\partial \theta} \mathbf{b} \end{bmatrix}.$$

Note that our goal is to obtain $\frac{\partial}{\partial \theta} \ell$. By chain rule, we obtain:

$$\frac{\partial}{\partial \theta} \ell = \begin{bmatrix} \nabla_{\mathbf{x}} \ell \\ \nabla_{\boldsymbol{\xi}} \ell \end{bmatrix}^\top \begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{x} \\ \frac{\partial}{\partial \theta} \boldsymbol{\xi} \end{bmatrix} = - \underbrace{\begin{bmatrix} \nabla_{\mathbf{x}} \ell \\ \nabla_{\boldsymbol{\xi}} \ell \end{bmatrix}^\top \begin{bmatrix} \mathbf{C} & \mathbf{F}^\top \\ \mathbf{F} & \mathbf{0} \end{bmatrix}^{-1}}_{\begin{bmatrix} \tilde{\mathbf{x}}^\top & \tilde{\boldsymbol{\xi}}^\top \end{bmatrix}} \underbrace{\begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{C}\mathbf{x} + \frac{\partial}{\partial \theta} \mathbf{F}^\top \boldsymbol{\xi} \\ \frac{\partial}{\partial \theta} \mathbf{F}\mathbf{x} - \frac{\partial}{\partial \theta} \mathbf{b} \end{bmatrix}}_{\mathbf{d}_\theta},$$

where we group the left two terms as $\begin{bmatrix} \tilde{\mathbf{x}}^\top & \tilde{\boldsymbol{\xi}}^\top \end{bmatrix} = \begin{bmatrix} -\nabla_{\mathbf{x}} \ell \\ -\nabla_{\boldsymbol{\xi}} \ell \end{bmatrix}^\top \begin{bmatrix} \mathbf{C} & \mathbf{F}^\top \\ \mathbf{F} & \mathbf{0} \end{bmatrix}^{-1}$ which are independent of the choice of θ and denote the remaining θ -dependent term as $\mathbf{d}_\theta = \begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{C}\mathbf{x} + \frac{\partial}{\partial \theta} \mathbf{F}^\top \boldsymbol{\xi} \\ \frac{\partial}{\partial \theta} \mathbf{F}\mathbf{x} - \frac{\partial}{\partial \theta} \mathbf{b} \end{bmatrix}$.

First, $\begin{bmatrix} \tilde{\mathbf{x}}^\top & \tilde{\boldsymbol{\xi}}^\top \end{bmatrix}$ is the solution to the following system:

$$\begin{bmatrix} \mathbf{C} & \mathbf{F}^\top \\ \mathbf{F} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{x}} \\ \tilde{\boldsymbol{\xi}} \end{bmatrix} = \begin{bmatrix} -\nabla_{\mathbf{x}} \ell \\ -\nabla_{\boldsymbol{\xi}} \ell \end{bmatrix},$$

which corresponds to the KKT condition of the following Lagrangian function:

$$\tilde{L} = \frac{1}{2} \tilde{\mathbf{x}}^\top \mathbf{C} \tilde{\mathbf{x}} + \nabla_{\mathbf{x}} \ell^\top \tilde{\mathbf{x}} + \tilde{\boldsymbol{\xi}}^\top (\mathbf{F} \tilde{\mathbf{x}} + \nabla_{\boldsymbol{\xi}} \ell). \quad (38)$$

⁴Note that although flipping signs in $\mathbf{F}$ and $\mathbf{b}$ yields an equivalent system, we choose the sign convention to be consistent with Eq. (22) and the KKT conditions in Eqs. (24)-(29) so that the signs of the optimal λ_t 's for both systems are aligned.

Since ℓ is only computed from $\mathbf{u}_1$, $\nabla_{\xi} \ell = \mathbf{0}$ and $\nabla_{\mathbf{x}} \ell^\top = [\mathbf{0}_{(T+1)d}^\top, \nabla_{\mathbf{u}_{out}} \ell^\top, \mathbf{0}_{(T-1)d}^\top]$. By rewriting $\tilde{\mathbf{x}} = [\tilde{\mathbf{h}}_0^\top, \dots, \tilde{\mathbf{h}}_T^\top, \tilde{\mathbf{u}}_1^\top, \dots, \tilde{\mathbf{u}}_T^\top]^\top$ and $\tilde{\xi} = [\tilde{\lambda}_0^\top, \dots, \tilde{\lambda}_T^\top]^\top$, the Lagrangian function in Eq. (38) corresponds to the LQR problem in Eq. (35).

Now we analyze $\mathbf{d}_\theta$. Suppose $\theta := \mathbf{A}_{t,ij}$, then $\frac{\partial}{\partial \theta} \mathbf{C} = \mathbf{0}$, $\frac{\partial}{\partial \theta} \mathbf{b} = \mathbf{0}$, and $[\frac{\partial}{\partial \theta} \mathbf{F}]_{td+i, (t-1)d+j} = 1$ with all other entries being zero. This gives

$$\frac{\partial \ell}{\partial \mathbf{A}_{t,ij}} = \begin{bmatrix} \tilde{\mathbf{x}}^\top & \tilde{\xi}^\top \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial \theta} \mathbf{F}^\top \xi \\ \frac{\partial}{\partial \theta} \mathbf{F} \mathbf{x} \end{bmatrix} = \xi_{td+i} \tilde{\mathbf{x}}_{(t-1)d+j} + \tilde{\xi}_{td+i} \mathbf{x}_{(t-1)d+j}.$$

Hence $\frac{\partial}{\partial \mathbf{A}_t} \ell = \lambda_t \tilde{\mathbf{h}}_{t-1}^\top + \tilde{\lambda}_t \mathbf{h}_{t-1}^\top$.

Suppose $\theta := \mathbf{B}_{t,ij}$, then $\frac{\partial}{\partial \theta} \mathbf{C} = \mathbf{0}$, $\frac{\partial}{\partial \theta} \mathbf{b} = \mathbf{0}$, and $[\frac{\partial}{\partial \theta} \mathbf{F}]_{td+i, (T+t)d+j} = 1$. Similarly, we have

$$\frac{\partial \ell}{\partial \mathbf{B}_{t,ij}} = \xi_{td+i} \tilde{\mathbf{x}}_{(T+t)d+j} + \tilde{\xi}_{td+i} \mathbf{x}_{(T+t)d+j}$$

In matrix form, it is equivalent to $\frac{\partial}{\partial \mathbf{B}_t} \ell = \lambda_t \tilde{\mathbf{u}}_t^\top + \tilde{\lambda}_t \mathbf{u}_t^\top$.

Likewise, consider $\theta := \mathbf{Q}_{t,ij}$ (or $\theta := \mathbf{R}_{t,ij}$), then $\frac{\partial}{\partial \theta} \mathbf{F} = \mathbf{0}$, $\frac{\partial}{\partial \theta} \mathbf{b} = \mathbf{0}$, and $[\frac{\partial}{\partial \theta} \mathbf{C}]_{td+i, td+j} = [\frac{\partial}{\partial \theta} \mathbf{C}]_{td+j, td+i} = 1/2$ (or $[\frac{\partial}{\partial \theta} \mathbf{C}]_{(T+t)d+i, (T+t)d+j} = [\frac{\partial}{\partial \theta} \mathbf{C}]_{(T+t)d+j, (T+t)d+i} = 1/2$). Henceforth, we can derive:

$$\frac{\partial \ell}{\partial \mathbf{Q}_{t,ij}} = \frac{1}{2} (\mathbf{x}_{td+i} \tilde{\mathbf{x}}_{td+j} + \tilde{\mathbf{x}}_{td+i} \mathbf{x}_{td+j}), \quad \frac{\partial \ell}{\partial \mathbf{R}_{t,ij}} = \frac{1}{2} (\mathbf{x}_{(T+t)d+i} \tilde{\mathbf{x}}_{(T+t)d+j} + \tilde{\mathbf{x}}_{(T+t)d+i} \mathbf{x}_{(T+t)d+j}),$$

i.e., $\frac{\partial}{\partial \mathbf{Q}_t} \ell = \frac{1}{2} (\mathbf{h} \tilde{\mathbf{h}}_t^\top + \tilde{\mathbf{h}}_t \mathbf{h}^\top)$ and $\frac{\partial}{\partial \mathbf{R}_t} \ell = \frac{1}{2} (\mathbf{u} \tilde{\mathbf{u}}_t^\top + \tilde{\mathbf{u}}_t \mathbf{u}^\top)$, respectively.

Finally, we examine $\theta := \mathbf{h}_{init,i}$. In this case, $\frac{\partial}{\partial \theta} \mathbf{C} = \mathbf{0}$, $\frac{\partial}{\partial \theta} \mathbf{F} = \mathbf{0}$, and $\frac{\partial}{\partial \theta} \mathbf{b}_i = 1$ while $\frac{\partial}{\partial \theta} \mathbf{b}_j = 0$ for $i \neq j$. Therefore, $\frac{\partial}{\partial \mathbf{h}_{init,i}} \ell = \xi_i$, then $\frac{\partial}{\partial \mathbf{h}_{init}} \ell = \tilde{\lambda}_0$. $\square$

B. Hardware Co-Designs and Optimization for TTC

In this section, we elaborate on the techniques to further improve the hardware efficiency of our LQR solver by taking advantage of Theorem 3.3. The pseudo-code for forward and backward passes are described in Algorithms 4 and 5.

Kernel Fusion. The algorithm implied by Theorem 3.3 is amenable to kernel fusion, since the dominant computation in the cumulative matrix product involves only basic tensor operations. We parallelize Eq. (12) by letting each kernel maintain a row block of $[\mathbf{Y}_1, \mathbf{Y}_2]$ and $\mathbf{y}_3$ on SRAM while loading Σ_t to perform matrix product in a loop. In fact, neither Σ_t nor $\mathbf{G}_t$ needs to be explicitly instantiated in HBM. Instead, Σ_t admits the following factorization:

$$\Sigma_t = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{B}_t \mathbf{R}_t^{-1} \mathbf{B}_t^\top & \mathbf{I} \end{bmatrix} \begin{bmatrix} (\mathbf{A}_t^{-1})^\top & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_t \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{Q}_{t-1} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}.$$

This allows the CUDA kernel to stream $\mathbf{B}_t$, $\mathbf{R}_t$, $\mathbf{A}_t$, and $\mathbf{Q}_{t-1}$ sequentially into on-chip SRAM to join the computation of each block in Σ_t . By fusing all tensor operations into a single kernel, our new solver further reduces HBM access and thereby alleviates the I/O latency. Finally, once $\mathbf{Y}_1$, $\mathbf{Y}_2$, and $\mathbf{y}_3$ are obtained, we leverage dense linear algebra routines in the cuBLAS library to compute $\mathbf{Y}_1^{-1} \mathbf{Y}_2$ and $\mathbf{Y}_1^{-1} \mathbf{y}_3$. Through a similar approach, we are further able to implement a CUDA kernel for the forward symplectic iteration described in Algorithm 3. Note that $\mathbf{S}_t$ follows a similar factorized form:

$$\mathbf{S}_t = \begin{bmatrix} \mathbf{I} & -\mathbf{B}_t \mathbf{R}_t^{-1} \mathbf{B}_t^\top \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{A}_t & \mathbf{0} \\ \mathbf{0} & (\mathbf{A}_t^{-1})^\top \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{Q}_{t-1} & \mathbf{I} \end{bmatrix},$$

allowing accumulating $\mathbf{S}_t$ via streaming $\mathbf{Q}_{t-1}$, $\mathbf{A}_t$, $\mathbf{R}_t$, and $\mathbf{B}_t$ in order. For gradient computation, we fuse two forward symplectic iterations for both primal and dual LQRs together, to efficiently obtain and combine trajectories $\{(\mathbf{h}_t, \mathbf{u}_t, \lambda_t)\}$ and $\{(\tilde{\mathbf{h}}_t, \tilde{\mathbf{u}}_t, \tilde{\lambda}_t)\}$ on chip without instantiating them on HBM.

Algorithm 4 Parallelized implementation of TTC layer’s forward pass

Require: $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$, initial state h_{init} , block size b .

Ensure: First-step actions u_{out} and caches for backward pass.

```

1: Initialize  $Y_1 \leftarrow I, Y_2 \leftarrow Q_T, Y_3 \leftarrow 0_{d \times d}$  on HBM.
2: for  $i = 1, \dots, \lceil d/b \rceil$  in parallel do
3:   Load on chip:  $Y_1^s \leftarrow Y_1[(i-1)b : ib, :]$ .
4:   Load on chip:  $Y_2^s \leftarrow \text{load } Y_2[(i-1)b : ib, :]$ .
5:   Initialize on chip:  $Y_3^s \leftarrow 0_{d \times d}$ .
6:   for  $t = T, \dots, 1$  do ▷ backward symplectic iteration
7:     Load and compute  $A_t, B_t, Q_t, R_t$  on chip.
8:      $Y_3^s \leftarrow \text{matmul}(\text{matmul}(Y_2^s, B_t), R_t^{-1})$ 
9:      $Y_1^s \leftarrow Y_1^s + \text{matmul}(Y_3^s, B_t^\top)$ 
10:     $Y_1^s \leftarrow \text{matmul}(Y_1^s, (A_t^{-1})^\top)$ 
11:     $Y_2^s \leftarrow \text{matmul}(Y_2^s, A_t)$ 
12:     $Y_2^s \leftarrow Y_2^s + \text{matmul}(Y_1^s, Q_{t-1})$ 
13:     $D^s \leftarrow \max(\text{norm}(Y_1^s, \text{dim} = 1), \text{norm}(Y_2^s, \text{dim} = 1))$  ▷ row-wise normalization
14:     $Y_1^s \leftarrow \text{div}(Y_1^s, D^s[:, \text{None}])$ 
15:     $Y_2^s \leftarrow \text{div}(Y_2^s, D^s[:, \text{None}])$ 
16:     $Y_3^s \leftarrow \text{div}(Y_3^s, D^s[:, \text{None}])$ 
17:   end for
18:   Write to HBM:  $Y_1[(i-1)b : ib, :] \leftarrow Y_1^s$ .
19:   Write to HBM:  $Y_2[(i-1)b : ib, :] \leftarrow Y_2^s$ .
20:   Write to HBM:  $Y_3[(i-1)b : ib, :] \leftarrow Y_3^s$ .
21: end for
22:  $(L, U) \leftarrow \text{lu\_factor}(Y_1)$ .
23:  $P \leftarrow \text{lu\_solve}(L, U, Y_2)$ .
24:  $\lambda_0 \leftarrow \text{matmul}(P, h_{init})$ .
25:  $u_1 \leftarrow \text{matmul}(R_1^{-1}, \text{matmul}(B_1, \lambda_0))$ .
26: Save  $(L, U), Y_3, \lambda_0$  for backward.
27: Write to HBM:  $u_{out} \leftarrow u_1$ .
    
```

Numerical Stability. Cumulative products of symplectic matrices may suffer from numerical instability, as their eigenvalues occur in reciprocal pairs (McDuff & Salamon, 2017). As a result, directly computing the symplectic iteration can cause the entries of the resulting matrices $[Y_1, Y_2]$ to grow exponentially, leading to numerical overflow and instability when computing Y_1^{-1} . To address this issue, we introduce a normalization scheme that keeps the values in $[Y_1, Y_2]$ within a controlled range. We observe that row-wise preconditioning of Y_1 and Y_2 does not affect the final result $Y_1^{-1}Y_2$. Based on this observation, we construct a diagonal matrix D with entries $D_{ii} = \max\{\|Y_{1,i}\|_1, \|Y_{2,i}\|_1\}$ and apply D^{-1} to $[Y_1, Y_2]$ immediately after each step. Although $[Y_1, Y_2]$ are partitioned row-wise and distributed across CUDA blocks, this normalization can be performed independently within each kernel without synchronization.

Mixed Precision. The TTC layer supports either full-precision (float32) or half-precision (float16 or bfloat16) inputs. Within the fused kernels of the reverse symplectic iteration, we deliberately keep all on-chip tensors Y_1 and Y_2 in float32 to preserve numerical accuracy during long-horizon matrix accumulations. The output tensor Y_1 also remains in float32, since LU decomposition is particularly sensitive to precision, whereas other intermediate quantities can be safely cast to lower precision when written back to HBM. In the forward symplectic iteration for gradient computation, all on-chip tensors are likewise maintained in float32 and only cast to the target precision when stored in HBM.

Caching for Backward. Taking a closer look at the dual LQR in Theorem 3.2, we observe that the coefficients associated with the affine terms are nonzero if and only if $t = 1$. Therefore, compared with the primal LQR, the symplectic iteration described in Theorem 3.3 differs only at the final step. This mathematical similarity allows several intermediate quantities computed during the forward pass to be reused in the backward pass: (1) The matrix Y_1 , required in both the primal and dual LQRs, is identical in the two cases. Hence, it can be cached after the forward symplectic iteration. In practice, we store its LU decomposition, since forward and backward computations only require applying Y_1^{-1} to Y_2 and y_3 , respectively. (2)

Algorithm 5 Parallelized implementation of TTC layer’s backward pass

Require: $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$, initial state h_{init} , gradient $\nabla_{u_{out}} \ell$ and caches from forward pass (L, U) , Y_3 , λ_0 .

Ensure: Gradients $\{(G_{A_t}, G_{B_t}, G_{Q_t}, G_{R_t})\}_{t=1}^T$ and $G_{h_{init}}$ on HBM.

- 1: Initialize $h \leftarrow h_{init}$, $\lambda \leftarrow \lambda_0$, $\tilde{h} \leftarrow h_{init}$ on chip.
- 2: Load (L, U) , Y_3 , λ_0 from forward cache.
- 3: $y_3 \leftarrow \text{matmul}(Y_3, -\nabla_{u_{out}} \ell)$.
- 4: Initialize $\tilde{\lambda} \leftarrow \text{lu.solve}(L, U, y_3)$ on chip.
- 5: Write to HBM: $G_{h_{init}} \leftarrow \tilde{\lambda}$.
- 6: **for** $t = 1, \dots, T$ **do** ▷ forward symplectic iteration
- 7: Load and compute A_t, B_t, Q_t, R_t on chip.
- 8: $\lambda \leftarrow \lambda - \text{matmul}(Q_{t-1}, h)$
- 9: $\tilde{\lambda} \leftarrow \tilde{\lambda} - \text{matmul}(Q_{t-1}, \tilde{h})$
- 10: $\lambda \leftarrow \text{matmul}((A_t^{-1})^\top, \lambda)$
- 11: $\tilde{\lambda} \leftarrow \text{matmul}((A_t^{-1})^\top, \tilde{\lambda})$
- 12: Write to HBM: $G_{A_t} \leftarrow \lambda \tilde{h}^\top + \tilde{\lambda} h^\top$.
- 13: $u \leftarrow -\text{matmul}(R_t^{-1}, \text{matmul}(B_t^\top, \lambda))$
- 14: $\tilde{u} \leftarrow -\text{matmul}(R_t^{-1}, \text{matmul}(B_t^\top, \tilde{\lambda}))$
- 15: **if** $t = 1$ **then**
- 16: $\tilde{u} \leftarrow \tilde{u} - \text{matmul}(R_t^{-1}, \nabla_{u_{out}} \ell)$
- 17: **end if**
- 18: Write to HBM: $G_{B_t} \leftarrow \lambda \tilde{u}^\top + \tilde{\lambda} u^\top$.
- 19: $h \leftarrow \text{matmul}(A_t, h) + \text{matmul}(B_t, u)$
- 20: $\tilde{h} \leftarrow \text{matmul}(A_t, \tilde{h}) + \text{matmul}(B_t, \tilde{u})$
- 21: Write to HBM: $G_{Q_t} \leftarrow (h \tilde{h}^\top + \tilde{h} h^\top)/2$.
- 22: Write to HBM: $G_{R_t} \leftarrow (u \tilde{u}^\top + \tilde{u} u^\top)/2$.
- 23: **end for**

Note that $y_3 = [I \quad Q_T] \left(\prod_{t=1}^{T-1} \Sigma_{T-t+1} \right) [0 \quad -B_T R_T^{-1} \nabla_{u_{out}} \ell]^\top$. The right block of $[I \quad Q_T] \left(\prod_{t=1}^{T-1} \Sigma_{T-t+1} \right)$ is already computed during the forward pass. This block can be cached and directly reused in the backward computation. By exploiting these caching strategies, we eliminate the need for an additional symplectic iteration during backpropagation. For gradient computation, we fuse two forward symplectic iterations in one GPU kernel for computing and combining trajectories $\{(h_t, u_t, \lambda_t)\}$ and $\{(\tilde{h}_t, \tilde{u}_t, \tilde{\lambda}_t)\}$ on chip without instantiating them on HBM. See Appendix B for more details.

C. Architectural Designs for TTC

Formulation of TTC Block. As mentioned in Sec. 4, TTC-Net employs a linear mapping to project context-rich token features produced by attention into the initial state of the TTC layer. The output of the TTC layer is then normalized and passed through another linear mapping, after which it is added back to the residual stream of the LLM backbone. Please refer to Fig. 4 for a quick overview. The entire block can be expressed as:

$$\begin{aligned} h_0 &= W_{in} \text{LN}(x_{in}), & \{(A_t, B_t, Q_t, R_t)\}_{t=1}^T &= \text{Cxt}(h_0, T), \\ o &= \text{TTC}(h_0, \{(A_t, B_t, Q_t, R_t)\}_{t=1}^T), & x_{out} &= x_{in} + W_{out} \text{LN}(o), \end{aligned}$$

where x_{in} is a token feature output by the preceding attention layer, x_{out} represents the final output, LN denotes a layer normalization, Cxt($\cdot$) is the contextualization mechanism introduced above, and W_{in} , W_{out} are input and output projection matrices.

Multi-Head Structure. Like many modern architectures, the TTC layer adopts a multi-head structure. The input state is partitioned into small blocks (e.g., of size 16), each of which generates a distinct set of TTC parameters for a corresponding head. For memory efficiency, we share the basis matrices $\{Q^{(i)}\}_{i=1}^r$ and $\{B^{(i)}\}_{i=1}^r$ as well as s_Q and s_B across all heads. The head-wise outputs are then concatenated and combined with a linear layer.

Parameterization. We parameterize the time-modulation coefficients $\Gamma_{\square}$, where $\square \in \{A, B, Q\}$, in the logarithmic domain. This improves numerical stability and efficiency when evaluating powers of the form $\Gamma_{\square}^t$. Specifically, we use a linear layer followed by a softplus activation, denoted $s_{\Gamma_{\square}}$, to generate the log-scale time-modulation coefficients. In the kernel, these are converted into time-dependent coefficients via $\exp(-t \cdot s_{\Gamma_{\square}}(\mathbf{h}_0))$. Furthermore, we observe that $\mathbf{R}_t$ appears only in its inverse form $\mathbf{R}_t^{-1}$. Therefore, rather than computing the matrix inverse at each time step, we directly let s_R output the diagonal of $\mathbf{R}_t^{-1}$ and use it as input to the LQR solver. During the backward, we directly compute gradients in terms of log-scale time-modulation coefficients and the inverse of $\mathbf{R}_t$.

Training Strategy. The time modulation strategy described above enables the TTC layer to support an arbitrary planning horizon T as input. However, training the TTC layer with a fixed horizon can induce distribution shift in downstream layers when a different horizon is used at test time. To mitigate this issue, we adopt a mixed-horizon training strategy. At each training iteration, we sample a random planning horizon from a truncated Poisson log-normal (PLN) distribution (Geiping et al., 2025): $\tau \sim \mathcal{N}(\log(T_{\mu}) - \frac{1}{2}T_{\sigma}^2, T_{\sigma}^2)$, $T_{train} \sim \text{Poisson}(\exp(\tau)) + 1$, where T_{μ} denotes the mean of T , and T_{σ} controls the standard deviation. To ensure T_{train} is bounded, we use rejection sampling: we repeatedly sample T_{train} until the drawn T_{train} falls within the range. In our experiments, we set the mean to $T_{\mu} = 8$, the log-scale standard deviation to $T_{\sigma} = 0.1$, and the maximum horizon to 32.

Test-Time Scaling. At test time, the planning horizon T_{test} can be adjusted arbitrarily. Increasing T_{test} causes the symplectic iteration to run longer and consume additional FLOPs; however, a longer horizon also enables the model to explore trajectories more deeply and emphasize long-term objectives, often leading to more accurate solutions. Consequently, the TTC layer can adaptively allocate additional computation to reason longer and plan over an extended horizon. This exposes a new test-time scaling (Snell et al., 2024) axis for reasoning that is native to, and intrinsically supported by, the architecture. We use experiments in Fig. 5 to demonstrate the test-time scaling potential of TTC-Net.

D. Discussion

More Memory-Based Architectures. Beyond simple linear regression, it is also viable to perform parametric regression with a non-linear M_t (e.g., letting M_t be a neural network) (Sun et al., 2024; Zhang et al., 2025; Tandon et al., 2025), a loss functions other than the ℓ_2 error (Behrouz et al., 2025c), a decaying or truncated $w_{t,\tau}$ (Gu et al., 2020; Behrouz et al., 2025a), different regularization terms (Von Oswald et al., 2023), more advanced optimization algorithms (Behrouz et al., 2024; 2025a; von Oswald et al., 2025; Peng et al., 2025b), such as Adam, Muon (Jordan et al., 2024), conjugate gradient, and Chebyshev iteration. Each such modification gives rise to a distinct memory-based architecture that is covered in Eq. (1). A comprehensive summary of these extensions can be found in Behrouz et al. (2025c). Another thread of work further combines multiple memory mechanisms, yielding hybrid sequence models (Ren et al., 2024; Dong et al., 2024; Zancato et al., 2024; Du et al., 2025).

Difference from Amos et al. (2018). Seminal work by Amos et al. (2018) proposed leveraging differentiable LQR as a trainable neural network layer for end-to-end control and planning for the first time. In contrast to Amos et al. (2018), we introduce a contextualized parameterization that adapts the underlying control problem dynamically to the input context. Moreover, while Amos et al. (2018) focuses on classical control tasks, our work applies an LQR-based module to enhance reasoning capabilities in LLMs. Finally, we propose a new scalable LQR solver that enables efficient computation of the LQR-based TTC layer, thereby making its integration into large-scale LLMs practical.

Comparison with Test-Time Discovery. TTD (Yuksekgonul et al., 2026) is a recent work that came out during the preparation of this paper. To the best of our knowledge, it is the only other approach that explores online RL-based adaptation at test time. However, the technical approaches differ substantially: TTD performs model-free, policy-based RL by updating the model parameters (i.e., slow weights), whereas our method adopts a model-based, value-based RL formulation that performs planning over the hidden states (i.e., fast weights) without modifying the model weights. We view these two approaches as complementary, and their integration may further enhance test-time planning and reasoning capabilities. We also make a table to illustrate the categories of test-time learning methods.

Connection to World Models. TTC generates the environment parameters $\{(\mathbf{A}_t, \mathbf{B}_t, \mathbf{Q}_t, \mathbf{R}_t)\}_{t=1}^T$ conditioned on the input context, and then solves a control problem defined over these parameters to produce the output. The resulting linear state transitions and quadratic cost together constitute a simplified world model (Ha & Schmidhuber, 2018; Hafner et al.,

	Fast weight	Slow weight
SSL	DeltaNet(Yang et al., 2024b), GDN (Yang et al., 2024a) Titans(Behrouz et al., 2024), Nested Learning (Behrouz et al., 2025b) MesaNet (von Oswald et al., 2025), GatedKalmanNet(Peng et al., 2025b)	TTT-MLP (Sun et al., 2024) E2E-TTT (Tandon et al., 2025)
RL	TTC (Ours)	TTD (Yuksekgonul et al., 2026)

Table 3. Taxonomy of various test-time learning methods.

2019; Hansen et al., 2022; 2023), capturing the essential dynamics and objectives relevant to the task. Rather than aiming for high-fidelity environment simulation, TTC emphasizes fast decision-making and learnability by solving this tractable world model efficiently and differentiating this system to train it in an end-to-end manner. From this perspective, TTC can be interpreted as an in-context world model: it infers a context-dependent and concise (solvable) representation of the “world”, upon which it performs planning and decision-making. All parameters of this “world” are then directly trained through the supervision on the predicted actions.

Connection to Continuous CoT. Continuous Chain-of-Thought (CoT) (Hao et al., 2024) has been proposed to enable LLMs to reason beyond discrete tokens by representing intermediate reasoning traces as latent vectors. The general framework of continuous CoT repeatedly invokes a transformer backbone without decoding intermediate representations into tokens, while maintaining reasoning entirely within the latent space. Following this paradigm, an extensive body of work has focused on architectural innovations. For example, Geiping et al. (2025) injects perturbations and incorporates initial signals as inputs to all recurrent blocks. Wang et al. (2025a) introduces a two-level loop structure with additional supervision applied at each recurrent block. Other works further improve the scalability of looped transformers (Zeng et al., 2025; Zhu et al., 2025b). On the theoretical side, Giannou et al. (2023); Yang et al. (2023a); De Luca & Fountoulakis (2024); Saunshi et al. (2025) show that looped transformers can significantly enhance expressivity and may even achieve Turing completeness. In addition, Zhu et al. (2025a) demonstrates that latent CoT can encode the search frontier more efficiently. TTC-Net shares a similar high-level structure in that the “thinking” process is modeled within a latent state space. However, unlike continuous CoT methods, which are typically simulating an open-ended search, TTC-Net employs a closed-loop reasoning process, where rewards incurred at later steps are propagated backward to earlier stages to guide decision-making.

E. More Experiments

E.1. Ablation Studies

We conduct an ablation study on the MATH-500 benchmark to evaluate the effect of different design choices in TTC-Net. We examine three key design dimensions: (1) time modulation, (2) training horizon sampling strategy, and (3) TTC layer insertion interval. As we described in Sec. 4, the full model is a combination of three key features: (1) time-heterogeneous parameterization, (2) training horizon sampled from a Poisson log-normal distribution, and (3) an 8:1 ratio of attention to TTC layers. Each ablation study varies one of these three design dimensions while keeping the remaining components the same as in the full configuration. All model variants are trained with a fixed time horizon $T_{train} = 8$. At evaluation time, we assess generalization to different inference-time horizons $T_{test} \in \{8, 16\}$. We adopt the same recipe in Sec. 5.2 to train each model and report the test accuracy on MATH-500. All results are shown in Tab. 4.

Time Homogeneity vs. Heterogeneity. Time-heterogeneous parameterization allows the matrices $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$ to vary across time steps t , whereas time-homogeneous parameterization adopts a shared set of parameters (A, B, Q, R) for all t . We implement the time-homogeneous variant by removing the time modulation coefficients. As shown in Tab. 4, the time-homogeneous TTC (rows 1-2) consistently underperforms its time-

 Table 4. **Ablation studies.** Accuracy (%) on MATH-500 dataset.

Variants	MATH-500
Base model	25.00
Parameterization	
Homo. ($T_{test} = 8$)	48.40
Homo. ($T_{test} = 16$)	45.70
Horizon sampling	
Fixed ($T_{test} = 8$)	50.60
Fixed ($T_{test} = 16$)	31.50
Unif. ($T_{test} = 8$)	50.80
Unif. ($T_{test} = 16$)	51.00
Interleaving ratio	
Attn:TTC = 4:1	53.00
Attn:TTC = 16:2	50.30
Attn:TTC = 16:1	47.20
Full model	
<i>Time Heter. + PLN + (8:1)</i>	
TTC-Net ($T_{test} = 8$)	52.80
TTC-Net ($T_{test} = 16$)	53.60

heterogeneous counterpart (rows 10-11). Moreover, when the planning horizon is changed at test time, performance further degrades under the homogeneous setting. These results support our hypothesis that time-heterogeneous dynamics $\{(A_t, B_t, Q_t, R_t)\}_{t=1}^T$ are critical for modeling complex latent-space dynamics. In contrast, time-uniform linear dynamics suffer from limited expressivity. Additionally, without the discounting effect of the time modulation, generalization across planning horizons deteriorates.

Horizon Sampling Strategy. We further consider two alternative training horizon sampling strategies besides the Poisson log-normal (PLN) distribution: (1) a fixed horizon with $T_{train} = 8$, and (2) a uniform distribution with $T_{train} \sim \text{Unif}(1, 32)$. According to Tab. 4, training with a fixed horizon yields comparable performance at the matched test horizon (row 3), but fails to generalize or further improve when evaluated with a larger test-time horizon (row 4). The uniform horizon sampling strategy (rows 5–6) achieves accuracy on par with the full model (rows 10–11). However, this comes at a substantially higher training cost. Larger planning horizons require increased computational time. The sampled horizon from a PLN distribution concentrates around the mean value of 8, whereas the uniform distribution nearly doubles the average training horizon.

Interleaving Pattern between Attention and TTC. We further study the interleaving ratio between attention and TTC layers. We use $\text{Attn:TTC} = n : m$ to denote that m TTC layers are inserted after every n attention layers. Our default configuration is an 8 : 1 ratio in a 32-layer Transformer. The results are reported in rows 7–9 of Tab. 4. We observe that increasing the number of TTC layers can monotonically improve overall accuracy (rows 7 and 9). However, this improvement comes at a higher computational cost and is less cost-effective than adopting a relatively smaller interleaving ratio while increasing the test-time horizon (rows 7 and 11). Moreover, comparing configurations that stack more TTC layers consecutively (row 8) with those that interleave them more uniformly with attention layers (row 10), the results (8 : 1 vs 16 : 2) indicate that distributing TTC layers evenly throughout the network yields better performance. These observations justify our default choice of 8 : 1.

E.2. Comparison with Decoding-Based Scaling

A common approach to improving reasoning at test time is to allocate additional decoding computation. Methods such as Best-of- N sampling (BoN) (Stiennon et al., 2020), self-consistency (Wang et al., 2022), Tree-of-Thoughts (ToT) (Yao et al., 2024), and planning-based decoding methods such as RAP (Hao et al., 2023) improve performance by sampling, filtering, or searching over candidate reasoning traces. Unlike TTC-Net, which incorporates planning as an internal architectural mechanism over latent space, these approaches instantiate planning externally: the base model remains unchanged, while additional inference-time procedures are applied on top of autoregressive generation.

To compare these two forms of test-time computation, we evaluate TTC-Net against BoN and ToT under the same reasoning setting. All methods use chain-of-thought prompting. Tab. 5 shows that TTC-Net achieves higher accuracy while generating substantially fewer tokens. The gap in output length is particularly significant: external search methods require more than 20K generated tokens on average, whereas TTC-Net produces more concise reasoning traces. This suggests that internal latent planning can improve reasoning quality without relying on extensive enumeration and filtering in the token space. Nevertheless, TTC is complementary to decoding-based scaling techniques, and they can be combined to achieve stronger performance. TTC differs in that the planning computation is amortized through the model architecture.

We also isolate the interaction between TTC and explicit chain-of-thought generation. To disable explicit CoT for LLMs, we enforce the following system prompt:

Table 5. Comparison with decoding-based test-time scaling methods. TTC-Net improves accuracy while requiring substantially fewer generated tokens.

Method	Acc.	Avg. Tokens
BoN	22.43	20731.10
ToT	21.08	22412.02
TTC-Net	23.34	2801.50

```

1 Solve math problems silently. Do not disclose chain-of-thought, scratch work, intermediate
   reasoning, or hidden deliberation.
2
3 Return only the final answer in this exact format:
4
5 \[
6 \boxed{\dots}
7 \]
```

Tab. 6 reports a 2×2 comparison on AMC, varying whether the model includes TTC layers and whether the inference prompt encourages explicit CoT. TTC improves performance in both regimes. Without explicit CoT, TTC increases accuracy from 3.92% to 7.83%, indicating that latent planning provides a meaningful internal reasoning even when token-space reasoning steps are suppressed. With CoT enabled, TTC further improves accuracy from 20.78% to 23.34%, showing that TTC and explicit reasoning traces are complementary.

We note that TTC should not be viewed as a replacement for chain-of-thought reasoning. Instead, TTC improves the latent computation underlying each generated reasoning step. Explicit CoT remains important for difficult problems, while TTC provides an additional internal planning mechanism that can reduce reliance on long or redundant decoding-time search.

E.3. Evaluation beyond Reasoning Benchmarks

Although our primary goal is to improve reasoning through architectural design, we also evaluate whether TTC-Net preserves general language modeling and understanding capabilities beyond math reasoning benchmarks. We evaluate the fine-tuned LLaMA-based checkpoints on standard LM evaluation benchmarks, including commonsense reasoning, reading comprehension, knowledge-intensive multiple-choice evaluation, and code generation. The results are reported in Tab. 7.

Table 7. **Evaluation beyond math reasoning benchmarks.** TTC-Net improves several reasoning-heavy and code-related tasks while maintaining competitive performance on general language understanding benchmarks.

Model	HellaSwag	WinoGrande	ARC-C	MMLU	HumanEval
LLaMA + SFT	54.01	74.74	51.79	59.31	32.32
TTC-Net	53.75	72.09	59.22	64.02	49.39

TTC-Net obtains gains on ARC-Challenge, MMLU, and HumanEval, which require long-horizon reasoning or program synthesis ability. On HellaSwag and WinoGrande, TTC-Net remains competitive with the raw SFT baseline, suggesting that the additional planning module preserves general language modeling ability learned in base models. These results support the generality of TTC beyond the math benchmarks used in our main evaluation.

E.4. Evaluation on Qwen Models

We further validate TTC-Net on a stronger reasoning backbone, Qwen2.5-Math-7B. TTC layers are modular and can be inserted into transformer-based architectures without relying on a specific backbone. As shown in Tab. 8, TTC-Net improves over standard full-parameter fine-tuning across all evaluated math benchmarks, confirming the consistency of our proposed methods.

Table 8. **Results on Qwen2.5-Math-7B.** TTC-Net consistently improves over standard fine-tuning on a stronger reasoning backbone.

Model	MATH-500	AMC	AIME24	AIME25
Qwen2.5-Math-7B	43.8	33.0	6.7	6.7
+ Fine-tuning	74.4	51.7	20.4	18.7
+ TTC-Net	75.6	55.4	22.9	20.8

E.5. Computational Analysis

TTC-Net is designed to preserve efficient inference with hardware-level optimization. A naive implementation of finite-horizon optimal control would introduce substantial overhead, especially when the planning horizon is large. TTC-Net avoids this bottleneck through a hardware-friendly LQR solver and fused CUDA kernels that reduce memory I/O traffic.

Tab. 9 reports inference throughput on an NVIDIA H100 GPU. Compared with a standard transformer, TTC-Net introduces a small overhead at short planning horizons. At $T = 8$, throughput decreases from 47.16 to 45.77 tokens/s. Increasing the horizon leads to a gradual reduction in throughput, but the model remains practical even at $T = 128$. This behavior reflects the intended accuracy-efficiency trade-off: larger horizons allocate more computation to latent planning, while the parallel solver and fused implementation keep the additional cost moderate.

Table 6. Ablation study between TTC and explicit CoT prompting on AMC.

Model	w/ CoT	w/o CoT
w/o TTC	20.78	3.92
w/ TTC	23.34	7.83

Table 9. Inference throughput of Transformer and TTC-Net on an NVIDIA H100 GPU.

Model	Tokens/s
Transformer	47.16
TTC-Net ($T = 8$)	45.77
TTC-Net ($T = 16$)	44.61
TTC-Net ($T = 64$)	41.43
TTC-Net ($T = 128$)	36.18

E.6. Analysis of Latent Trajectories

The central hypothesis behind TTC-Net is that the explicit control mechanism can internalize meaningful planning trajectories. In this section, we testify this hypothesis by analyzing TTC latent states and measuring how they align with the explicit reasoning process specific to targeted tasks.

Linear Probing on Sudoku. We study whether TTC latent states become progressively more informative over the planning horizon. On Sudoku, we train a linear probe to decode intermediate TTC latent states into per-cell digit predictions. The TTC states themselves are not directly supervised with cell-level labels during model training; only the final outcome is used to make the final prediction. Tab. 10 shows that cell-level decoding accuracy increases with the planning step, from 42.50% at step 0 to 50.38% at step 8. This trend indicates that TTC planning progressively refines the latent representation toward the terminal solution.

Table 10. Linear probing accuracy of TTC latent states on Sudoku.

Step	Cell Acc.
0	42.50
2	42.96
4	46.20
8	50.38

Latent Trajectory Alignment with CoT. We measure the alignment between TTC latent trajectories and CoT token representations. Let $\mathbf{X} \in \mathbb{R}^{L \times d}$ denote token representations extracted from explicit CoT traces at the corresponding residual stream layer, where L is the CoT length. Likewise, we let $\mathbf{H} \in \mathbb{R}^{T \times d}$ denote the latent trajectory produced by TTC over a planning horizon of length T . Note that we extract $\mathbf{H}$ from the first token after the prompt. Because TTC planning steps and CoT tokens need not align one-to-one, we compute a monotone-path alignment score:

Table 11. Trajectory-level alignment between TTC latent dynamics and explicit CoT representations.

Pairing	Similarity
Same question	0.534
Random pairs	0.212

$$\text{sim}(\mathbf{H}, \mathbf{X}) = \max_{p \in \mathcal{P}} \frac{1}{|p|} \sum_{(i,j) \in p} \frac{\mathbf{H}_i^\top \mathbf{X}_j}{\|\mathbf{H}_i\|_2 \|\mathbf{X}_j\|_2}, \quad (39)$$

where $\mathcal{P} = \{((i_1, j_1), \dots, (i_m, j_m)) | (i_1, j_1) = (1, 1), (i_m, j_m) = (T, L), (i_{k+1} - i_k, j_{k+1} - j_k) \in \{(1, 0), (0, 1), (1, 1)\}\}$ denotes the set of monotone alignment paths between TTC planning steps and CoT token positions. The similarity score above intends to match each TTC latent to a consecutive chunk of tokens’ representations, and the maximization problem can be solved via dynamic programming. We compare this similarity score between latent trajectories and CoT representations derived from the same question and different question pairs. Tab. 11 reports the resulting similarity on AMC. TTC latent trajectories are more aligned with CoT representations from the same question than with randomly paired trajectories. This suggests that the TTC dynamics are highly correlated with the explicit reasoning trace, internalizing token-space CoT with a more compact latent space.

E.7. Experiment Details

Sudoku Solving. All experiments use a 32-layer model with 4 attention heads and an embedding dimension of 128, trained using a batch size of 16 and a learning rate of 5×10^{-3} with 10% tokens for warm-up and decay to 5×10^{-4} . The proposed TTC layer operates with a hidden and control dimension $d = 16$, employs 4 parallel TTC heads, and uses $r = 16$ for basis matrices $\{\mathbf{Q}^{(i)}\}_{i=1}^r$ and $\{\mathbf{B}^{(i)}\}_{i=1}^r$. Models are trained and evaluated on standard reasoning datasets, with 9,000 training samples and 1,000 test samples, following consistent data splits across experiments. We further provide a pseudo-code in Algorithm 6 to illustrate the multi-step completion process of a Sudoku game.

Math Reasoning. For math reasoning tasks, all models are fine-tuned using AdamW with a global batch size of 96, learning rate of 2×10^{-5} , weight decay of 0.1, and a cosine learning rate scheduler using 10% tokens for warm-up and then decaying to 2×10^{-6} . Each TTC layer has 16 heads, with each head operating on a state space of dimension 16 ($d = 16$). We set the number of basis to $r = 16$ for both $\{\mathbf{Q}^{(i)}\}_{i=1}^r$ and $\{\mathbf{B}^{(i)}\}_{i=1}^r$. TTC training employs stochastic planning horizons sampled from a Poisson–lognormal distribution, with a mean horizon of 8 and support ranging from 1 to 32 steps. During evaluation, we sample 8 solutions for each problem in AIME and AMC, using a temperature of 0.6. For Math-500, we employ greedy decoding to generate a single solution per problem.

Reasoning Data Curation. The curated data collection we used for fine-tuning models comprises questions from recent (2024–2025) verifiable reasoning datasets spanning mathematics, multi-domain academic subjects, science, medicine,

Algorithm 6 Multi-step Sudoku completion

Require: An incomplete board $B_{init} \in \{\text{[mask]}, 1, \dots, 9\}^{9 \times 9}$.

Ensure: Completed board $B \in \{1, \dots, 9\}^{9 \times 9}$.

```

1: Initialize board  $B \leftarrow B_{init}$ .
2: while  $\exists(i, j)$  such that  $B_{ij} = \text{[mask]}$  do
3:   Initialize prediction  $P \in \{1, \dots, 9\}^{9 \times 9}$ , confidence  $C \in [0, 1]^{9 \times 9} \leftarrow \mathbf{0}$ .
4:   Obtain per-cell digit probabilities:  $X \leftarrow \text{Predictor}(B) \in (0, 1)^{9 \times 9 \times 9}$ . ▷ Invoke neural network backbone
5:   for each masked cell  $(i, j)$  such that  $B_{ij} = \text{[mask]}$  do
6:      $P_{ij} \leftarrow \arg \max_k X_{ijk}$  ▷ Most likely digit for cell  $(i, j)$ 
7:      $C_{ij} \leftarrow \max_k X_{ijk}$  ▷ Confidence of the prediction
8:   end for
9:   Select most confident masked cell:  $(i^*, j^*) \leftarrow \arg \max_{(i, j)} C_{ij}$ .
10:  Fill selected cell with its prediction:  $B_{i^*, j^*} \leftarrow P_{i^*, j^*}$ .
11: end while
12: Return  $B$ .
```

finance, and coding, totaling on the order of 4 million objectively checkable QA instances. One of the primary motivations for selecting more recent datasets is to mitigate the spurious performance caused by potential data leakage and contamination, a common concern in large-scale industrial pre-trained models (Shojaee et al., 2025). A substantial portion is math-centric (e.g., DeepMath-103K, Big-Math, Massive-Math-455K, ORZ-72k, DeepScaleR), emphasizing problems with unique numeric or symbolic answers suitable for rule-based verification and reinforcement learning. Broader datasets (e.g., Multi-Subject-RLVR, II-Thought, Natural Reasoning, General-Reasoner, Nemotron-CrossThink) extend coverage to physics, social sciences, law, and engineering, often filtering for short, automatically verifiable answers. Domain-specific subsets target high-stakes fields such as medicine (MedReason, Medical-O1, Medical-O1-SFT) and finance (e.g., Fino1). Building on these QA pairs, we further generate chain-of-thoughts (CoT) using gpt-oss for those whose intermediate reasoning traces are not given. To ensure correctness, we retain only those CoTs that pass 32 independent rounds of verifications by gpt-oss. After deduplication, we randomly sample and keep 800K examples as the final curated SFT dataset.

Efficiency Benchmark. In Fig. 3, we evaluate computational efficiency on an NVIDIA H200 GPU. Both throughput and memory footprint are measured over a complete forward and backward pass. Throughput is measured in GB/s, reporting the median runtime together with the 20-80% percentile statistics to account for runtime variability. We benchmark two scaling dimensions: (1) planning horizon $T \in \{2^4, \dots, 2^{11}\}$ with logarithmic spacing, and (2) batch size $B \in \{2^4, \dots, 2^{13}\}$. Across all results, we set the state dimension $d = 16$. Throughput is computed based on the theoretical FLOPs BTd^3 of solving B -many batched d -dimensional LQR problem with horizon T via Riccati iteration and normalized by wall-clock latency. Memory footprint is evaluated separately by measuring peak GPU memory usage during execution, reported in GB. For memory benchmarking, we fix either $B = 1024$ (when varying T) or $T = 64$ (when varying B) to isolate scaling effects. All benchmarks compare different implementation providers under identical input generation and precision settings, and out-of-memory cases are recorded as zero throughput.